

High-performance sequential quasi-Monte Carlo with applications to football match modelling

Ryan TAN Jun Xiong

Student number: u5754603

Supervisor: Dr. Adrien Corenflos

September 14, 2026

ST980 MSc in Statistics Dissertation

Abstract

Sequential quasi-Monte Carlo (SQMC) is an extension of sequential Monte Carlo (SMC) in which independent Monte Carlo inputs are replaced by randomised low-discrepancy point sets and particles are ordered before resampling; under regularity conditions it can attain lower integration error than SMC, up to the Monte Carlo rate of $O_P(N^{-1/2})$. However, SQMC is more computationally expensive than SMC at $\mathcal{O}(N \log N)$ and its advantage may deteriorate as the effective dimension grows. We develop an implementation of SQMC on a GPU using `jax`, and measure its accelerator performance: on a linear-Gaussian filtering model an A100 GPU executed SQMC 6.2–13.4 times faster than a CPU at 2,048 particles across the five dimensions. We then demonstrate how Rao-Blackwellisation can be used to tackle high-dimensional limitations of SQMC in the context of football matches. This particle approach does not, however, outperform a factorial state-space model with an extended Kalman filter (EKF) approximation, indicating that the EKF is a suitable approximation for this problem. This work is part of a contribution to the open-source state-space inference package `cuthbert`.

Keywords: Sequential Monte Carlo, Sequential quasi-Monte Carlo, Low-discrepancy point sets, Hilbert sort, Rao-Blackwellisation, Parallel accelerators, Football

Acknowledgements

I would like to express my gratitude to my supervisor Adrien Corenflos, whose wisdom and guidance throughout the dissertation has been invaluable. I would also like to thank my parents, Joseph and Kim, who have supported me wholeheartedly throughout this journey.

Contents

Abstract	ii
Acknowledgements	iii
1 Introduction	1
2 High-performance sequential quasi-Monte Carlo (SQMC)	3
2.1 Background: SMC, QMC, and SQMC	3
2.1.1 Sequential Monte Carlo (SMC)	4
2.1.2 Quasi-Monte Carlo (QMC)	6
2.1.3 Sequential quasi-Monte Carlo (SQMC)	8
2.1.4 Key Properties of SQMC	10
2.2 Empirical evaluation: accelerator performance of the SQMC imple- mentation	11
2.2.1 Experimental design and measurement	12
2.2.2 QMC generation	12
2.2.3 Hilbert sorting	13
2.2.4 SQMC on CPU and GPU at a fixed iteration count	14
3 Rao-Blackwellised SQMC for Football Matches	17
3.1 Background	17
3.2 Model	20
3.3 Rao-Blackwellised SMC (RB-SMC)	21

3.4	RB-SMC Smoothing	25
3.4.1	Deriving the one-step backward kernel	26
3.4.2	The RB-SMC smoothing algorithm	27
3.5	Rao-Blackwellised SQMC (RB-SQMC)	28
3.6	Parameter Estimation	30
3.7	Performance comparison	32
3.7.1	Experimental design	32
3.7.2	Training: marginal log-likelihood	32
3.7.3	Prediction outputs	34
3.7.4	Evaluation	36
3.7.5	Comparison with the FIFA ranking	38
3.7.6	Discussion	39
4	Conclusion	41
A	Proofs	43
A.1	Kronecker structure reduces the inference dimension	43
A.2	The deterministic inverse-CDF transform in RB-SQMC	44
B	Code	47
B.1	Replication	47
C	Configuration	50
C.1	Data and split	50
C.2	Model parameters	51
C.3	Optimisation	51
C.4	Software environment	51
C.5	Full configuration	51
D	Prediction	53

D.1	Round of 32	54
D.2	Round of 16	55
D.3	Quarter-finals	55
D.4	Semi-finals	55
D.5	3rd place	56
D.6	Final	56
E	Rankings	57
E.1	Pre-tournament rankings	57
E.2	Post-tournament rankings	58

Chapter 1

Introduction

Sequential Quasi-Monte Carlo (SQMC) is an extension of the Sequential Monte Carlo (SMC) algorithm, replacing the Monte Carlo proposal with deterministic propagation through low-discrepancy point sets. The error rate of SQMC is smaller than the Monte Carlo rate of $\mathcal{O}_P(N^{-1/2})$, and we can expect the algorithm to reach faster convergence for a smaller class of functions (Chopin and Gerber, 2015). Furthermore, recent work suggests that, for a fixed particle count, the error of a standard SMC exceeds any fixed threshold at infinitely many time instants with probability equal to one, while SQMC’s error vanishes uniformly in time as N grows (Gerber, 2026). This safety guarantee, on a linear-Gaussian filtering problem, encourages the usage of SQMC over SMC in long-horizon problems.

However, the computational cost of SQMC is $\mathcal{O}(N \log N)$, where N is the number of simulations at each iteration, a significant increase over SMC, which is $\mathcal{O}(N)$. This overhead stems from two additional steps in SQMC—sorting the particles along a Hilbert curve before resampling and applying the inverse-CDF transform to the quasi-random points. Another issue with SQMC is that its performance advantage over SMC decreases in increasing dimensions. The most likely factor is the curse of dimensionality that affects SMC (Chopin and Gerber, 2017). Importance sampling, a key component of the SMC algorithm, suffers from weight degeneracy in high dimensions: the larger the dimension, the greater the discrepancy between the proposal and the target distribution, so that a small fraction of particles carry non-negligible weight at each step (Snyder et al., 2008; Bickel et al., 2008).

The dissertation makes two contributions. First, an implementation of the SQMC algorithm that is computationally efficient by leveraging accelerator hardware, specifically a GPU, using `jax`; the approach is generalisable to other similar parallel devices.

We then show that this implementation outperforms current CPU implementations of the same algorithms. Second, we demonstrate how the high-dimensional limitations of SQMC can be overcome by exploiting the sparse observation structure of football matches. By modelling the transition distribution as linear-Gaussian, we can perform Rao-Blackwellisation to integrate out the latent states of non-playing teams, while performing SQMC on the smaller subset of playing teams, thereby reducing the 'effective dimension' of the model and mitigating the weight degeneracy problems as described above.

All code and experiments are available in the public repository `ryantjx/sqmc-cut` `hbert`; as part of an ongoing process of integrating the SQMC algorithm into the `cuthbert` project under issue #255.

Chapter 2

High-performance sequential quasi-Monte Carlo (SQMC)

2.1 Background: SMC, QMC, and SQMC

Let $X_t \in \mathcal{X} \subseteq \mathbb{R}^d$ be a latent Markov state with observations Y_t . Let m_0 be a density on $\mathbb{R}^d$ for the initial state X_0 , let $m_t(\cdot \mid x_{t-1})$ be a transition density on $\mathbb{R}^d$ conditional on x_{t-1} , and let $g_t(\cdot \mid x_t)$ be an observation density on the observation space conditional on x_t ; all densities are with respect to Lebesgue measure. The joint density of the state-space model is given by

$$p(x_{0:T}, y_{1:T}) = m_0(x_0) \prod_{t=1}^T m_t(x_t \mid x_{t-1}) g_t(y_t \mid x_t) \quad (2.1)$$

One common inference task in state-space models is *filtering*—the task of recovering the distribution $p(x_t \mid y_{1:t})$, which is the distribution of the current latent state x_t given the data collected up to time t , $y_{1:t} = (y_1, \dots, y_t)$. Another common inference task is to learn the fixed parameter θ of a state-space model, which could be done via *smoothing*, recovering $p(x_{0:T} \mid y_{1:T})$ or via likelihood evaluation $p(y_{1:T})$. For a linear-Gaussian state-space model, these integrals are available analytically via the Kalman filter and the Rauch–Tung–Striebel smoother (Kalman and Bucy, 1961; Rauch et al., 1965). However, for non-linear or non-Gaussian state-space models, these integrals are intractable. Sequential Monte Carlo is a popular class of algorithms to approximate sequential intractable integrals in state-space models.

2.1.1 Sequential Monte Carlo (SMC)

Sequential Monte Carlo (SMC, also known as particle filtering) is a class of algorithms for computing recursively Monte Carlo approximations of a sequence of filtering distributions $p(x_t \mid y_{1:t})$, $t \in \{0, \dots, T\}$. The algorithm is based on the idea of performing importance sampling sequentially to approximate the sequence of distributions, known as Sequential Importance Sampling (SIS): choose an initial proposal density $q_0(x_0)$ and a sequence of proposal kernels $q_t(x_t \mid x_{t-1})$, $t \geq 1$, then simulate N times iteratively from these proposals and reweight the particles x_t^n with importance weights $\tilde{w}_t^n$

$$\tilde{w}_0^n = G_0(x_0^n), \quad \tilde{w}_t^n = \tilde{w}_{t-1}^n \cdot G_t(x_{t-1}^n, x_t^n), \quad w_t^n = \frac{\tilde{w}_t^n}{\sum_{m=1}^N \tilde{w}_t^m}$$

and the weight functions G_t are defined as

$$G_0(x_0) = \frac{m_0(x_0)}{q_0(x_0)}, \quad G_t(x_{t-1}, x_t) = \frac{m_t(x_t \mid x_{t-1})g_t(y_t \mid x_t)}{q_t(x_t \mid x_{t-1})}$$

We then obtain a consistent estimate of the filtering expectation $\mathbb{E}_{\pi_t}[\psi(x_t)]$ as $N \rightarrow \infty$.

$$\mathbb{E}_{\pi_t}[\psi(x_t)] \approx \hat{\psi}_t^N = \sum_{n=1}^N w_t^n \psi(x_t^n)$$

However, it is well known that SIS suffers from weight degeneracy—more particles get negligible weight over time. Gordon et al. (1993) introduced the additional step of *resampling*: drawing N times with replacement from the current set of particles proportional to their importance weights w_t^n . Particles with low weights get quickly discarded, while particles with large weights may get many children at the next iteration. Resampling controls the growth of the weight variance: with resampling at every step, the variance of filtering estimates typically remains stable over time, whereas without resampling it can grow rapidly over successive steps (Del Moral and Guionnet, 1999; Chopin and Papaspiliopoulos, 2020).

Resampling draws the ancestor indices $a^{1:N}$ from the discrete distribution with weights $w^{1:N}$. The inverse-CDF construction with independent uniform inputs $u^{1:N} \sim \mathcal{U}[0, 1]$ is known as multinomial resampling (Devroye, 1986, pg. 214): given the normalised weights and the uniforms, one computes the cumulative sums of the weights and maps each uniform to the smallest index whose cumulative sum is at least u^n . We

will re-use this inverse-CDF construction in SQMC.

Algorithm 1 Multinomial resampling

Require: normalised weights $w^{1:N}$ and uniforms $u^{1:N} \sim \mathcal{U}[0, 1]$

- 1: Compute the cumulative sums $C^n = \sum_{m=1}^n w^m$ for $n = 1, \dots, N$.
 - 2: **for** $n = 1, \dots, N$ **do**
 - 3: Set $a^n = \min\{m : C^m \geq u^n\}$.
 - 4: **end for**
 - 5: **return** ancestor indices $a^{1:N}$.
-

A simplification can be made to the SIS algorithm by setting the proposal kernels to the transition density, $q_t(x_t | x_{t-1}) = m_t(x_t | x_{t-1})$, the weight function then reduces to the observation likelihood,

$$G_t(x_{t-1}, x_t) = g_t(y_t | x_t)$$

so that the transition density need not be evaluated when computing the weights. This simplification and resampling yields the bootstrap particle filter of Gordon et al. (1993) defined in Algorithm 2, whose per-iteration cost is $\mathcal{O}(N)$. This filter is the basis of the Rao-Blackwellised construction in Section 3.3.

Algorithm 2 Bootstrap particle filter

Require: model densities m_0, m_t, g_t ; observations $y_{1:T}$; particle count N ; proposals q_0, q_t ; uniforms for resampling

Ensure: weighted particle approximation of $p(x_t | y_{1:t})$ and normalising-constant estimate $\hat{Z}_t$ at each t

- 1: Sample $x_0^n \sim q_0$ and set $\tilde{w}_0^n = G_0(x_0^n)$ for $n = 1, \dots, N$.
 - 2: **for** $t = 1, \dots, T$ **do**
 - 3: Resample: draw ancestor indices a_t^n from the discrete distribution with weights $w_{t-1}^{1:N}$. See Algorithm 1
 - 4: Propagate: sample $x_t^n \sim q_t(\cdot | x_{t-1}^{a_t^n})$ for $n = 1, \dots, N$.
 - 5: Weight: compute $\tilde{w}_t^n = G_t(x_{t-1}^{a_t^n}, x_t^n)$ and normalise $w_t^n = \tilde{w}_t^n / \sum_{m=1}^N \tilde{w}_t^m$.
 - 6: **end for**
 - 7: **return** weighted particles $\{w_t^n, x_t^n\}_{n=1}^N$ and $\hat{Z}_t = \prod_{s=0}^t \left[\frac{1}{N} \sum_{n=1}^N \tilde{w}_s^n \right]$.
-

2.1.2 Quasi-Monte Carlo (QMC)

QMC (Quasi-Monte Carlo) is an alternative to Monte Carlo (MC) methods for approximating integrals. The idea is to replace Monte Carlo random samples with a low-discrepancy point set $\{\mathbf{u}^n\}_{n=1}^N \subset [0, 1]^d$ to estimate our integral of interest,

$$\mathbb{E}[f(U)] = \int_{[0,1]^d} f(\mathbf{u}) d\mathbf{u} \approx \frac{1}{N} \sum_{n=1}^N f(\mathbf{u}^n), \quad U \sim \mathcal{U}[0, 1]^d$$

where $\mathbf{u}^n$ is the n -th member of a low-discrepancy point set. Informally, the points of such a set are spread more evenly over the unit cube than random points, so the empirical distribution of the set departs less from uniformity. The quality of a point set is quantified by its *discrepancy*, a measure of how far its empirical distribution differs from uniformity (Niederreiter, 1992). The relevance of discrepancy to integration is given by the Koksma–Hlawka inequality, which bounds the integration error by the product of the variation of the integrand and the discrepancy of the point set,

$$\left| \frac{1}{N} \sum_{n=1}^N f(\mathbf{u}^n) - \int_{[0,1]^d} f(\mathbf{u}) d\mathbf{u} \right| \leq V(f) D^*(\{\mathbf{u}^n\}_{n=1}^N)$$

where $V(f)$ is the variation of f in the sense of Hardy and Krause and D^* is the star discrepancy (Niederreiter, 1992). The inequality requires bounded Hardy–Krause variation; for such integrands, low-discrepancy point sets reduce the bound on the integration error, and QMC can achieve an error close to $\mathcal{O}(N^{-1})$, compared with the Monte Carlo rate $\mathcal{O}_P(N^{-1/2})$. Figure 2.1 illustrates the difference between the two constructions in two dimensions: the random points cluster and leave gaps, whereas the Sobol’ points cover the square more evenly.

Randomised quasi-Monte Carlo (RQMC)

RQMC randomises a QMC point set while preserving its low-discrepancy structure. The first advantage of RQMC over QMC is that it yields an unbiased estimator for our integral of interest which it possible to evaluate the approximation error through independent simulations.

$$\mathbb{E}_{\mathcal{R}} \left[\frac{1}{N} \sum_{n=1}^N f(\mathbf{u}^n) \right] = \int_{[0,1]^d} f(\mathbf{u}) d\mathbf{u}$$

This unbiased property carries over to SQMC: when the point sets are randomised,

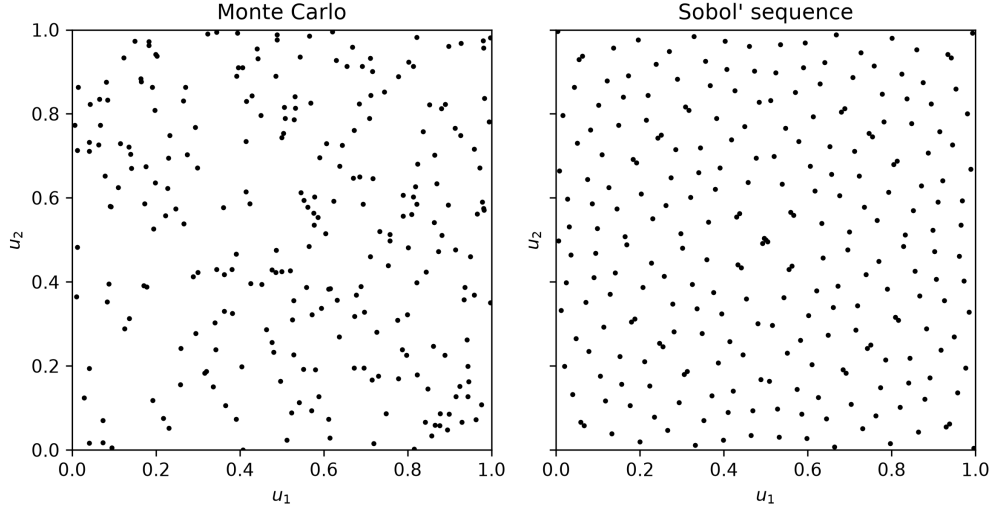


Figure 2.1: Two-dimensional point sets of size $N = 256$: independent uniform random points (left) and the first 256 non-origin points of the Sobol' sequence (right).

SQMC provides an unbiased estimator of the normalising constant Z_t , and hence of the likelihood of the state-space model (Chopin and Gerber, 2015; Del Moral, 1996). The second advantage of RQMC over QMC concerns variance. For digital nets and sequences in base b under a scrambling scheme that preserves their net structure, such as Owen's nested scrambling, the variance of the RQMC estimator is $\mathcal{O}(N^{-2}(\log N)^{d-1})$ for square-integrable integrands (Owen, 1997). Under the stronger nested scrambling of Owen, the mean squared error (MSE) of RQMC improves to $\mathcal{O}(N^{-3+\epsilon})$ for sufficiently smooth integrands and $\mathcal{O}(N^{-2+\epsilon})$ for non-smooth integrands of bounded variation, for any $\epsilon > 0$ (Owen, 1997, 1998). Both rates are a significant improvement over the Monte Carlo rate of $\mathcal{O}(N^{-1})$.

QMC Sequences

We introduce two popular low-discrepancy sequences, both of which can be randomised as described in Section 2.1.2. The *Halton sequence* (Halton, 1960) uses a radical-inverse sequence (reversing the base- b digits of the index) in a distinct prime base for each dimension. If $n = \sum_{k \geq 0} a_k(n)b^k$, where n is the index of the points in the Halton sequence and b is the base, then the radical-inverse function ϕ_b is defined as

$$\phi_b(n) = \sum_{k \geq 0} a_k(n)b^{-k-1}, \quad \mathbf{u}^n = (\phi_{b_1}(n), \dots, \phi_{b_d}(n)) \in [0, 1]^d$$

The Halton sequence is simple to construct, but its coordinates in higher dimensions

use successively larger prime bases and become strongly correlated, which degrades uniformity.

The *Sobol' sequence* (Sobol', 1967) is a digital (t, s) -sequence in base 2, constructed from direction numbers.¹ A (t, s) -sequence is a sequence of points in $[0, 1]^s$, and t is a non-negative integer that measures its uniformity: smaller t means the points are spread more evenly, with $t = 0$ optimal. If $a(n)$ is the binary digit vector of n and C_j is the generator matrix for dimension j , then

$$z_j = C_j a(n) \pmod{2}, \quad u_j^n = \sum_{r=1}^m z_j 2^{-r}, \quad \mathbf{u}^n = (u_1^n, \dots, u_d^n) \in [0, 1]^d$$

This multiplication is a bitwise XOR operation, and the Sobol' sequence can be generated efficiently with bit-level operations. The Sobol' sequence has good low-dimensional projections and is often the standard choice for high-dimensional QMC. However, it is limited to dimensions $d \leq 1111$ with the direction numbers of Joe and Kuo (2008). Moreover, our Sobol' implementation uses a left linear matrix scramble followed by a digital shift (LMS+shift) (Matoušek, 1998), which is cheaper than nested scrambling but attains only the general RQMC variance rate $\mathcal{O}(N^{-2}(\log N)^{d-1})$ (Owen, 1997), rather than the faster rates $\mathcal{O}(N^{-3+\epsilon})$ and $\mathcal{O}(N^{-2+\epsilon})$ available to nested scrambling for smooth and non-smooth integrands respectively.

Both sequences are implemented in our codebase, but we use the Sobol' sequence as the default for the SQMC experiments because of its favourable projections and its efficient bit-level generation.

2.1.3 Sequential quasi-Monte Carlo (SQMC)

Sequential quasi-Monte Carlo (SQMC) combines SMC and QMC. It is an extension of the array-RQMC algorithm of L'Ecuyer et al. (L'Ecuyer et al., 2006). Because the particles are propagated deterministically from a low-discrepancy point set, the resampling step of SMC cannot be applied directly: drawing the ancestors by a stochastic inverse-CDF transform or other resampling algorithms would destroy the low-discrepancy structure of the particles. SQMC instead sorts the particles before resampling, so that the inverse-CDF transform acts on an ordered set and preserves

¹A digital sequence is generated by linear algebra over the field of two elements: each coordinate is obtained by multiplying the binary digit vector of the point index by a generator matrix, an operation that reduces to bitwise XOR. The direction numbers are the columns of these generator matrices; they are fixed constants that define a particular Sobol' sequence, and their choice controls its uniformity.

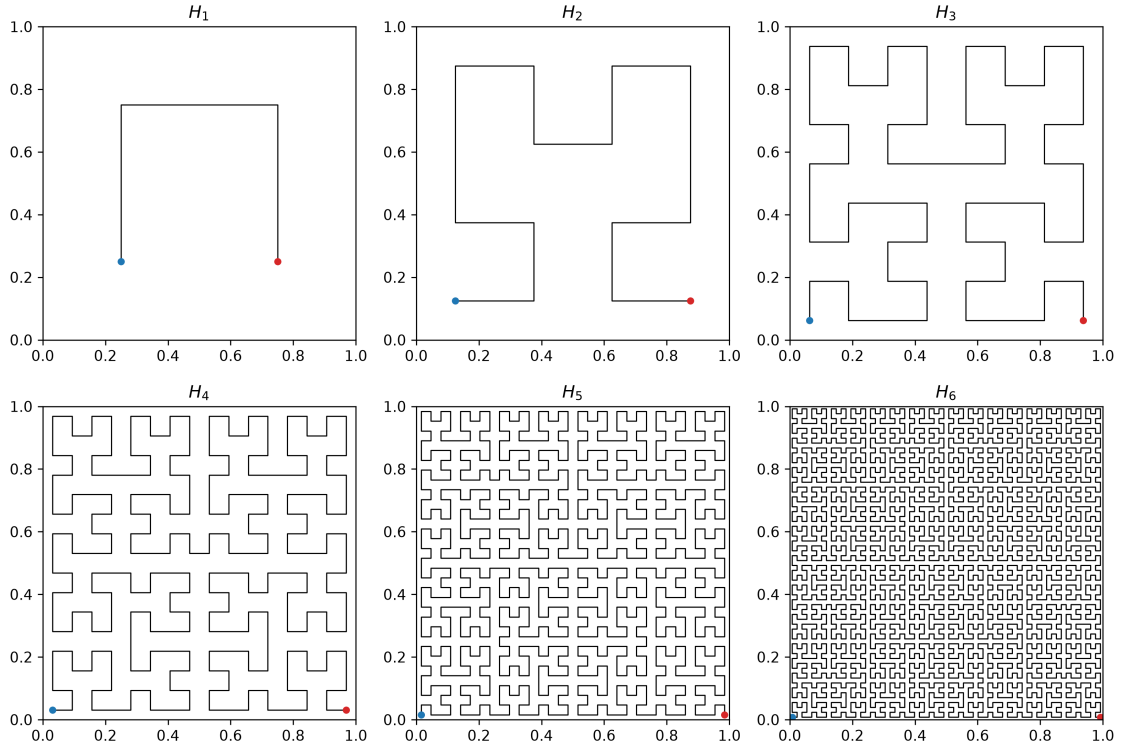


Figure 2.2: First six iterates $H_1, \dots, H_6$ of the two-dimensional Hilbert space-filling curve, showing the curve filling the unit square at increasing resolution. Start and end points are marked in blue and red.

the low-discrepancy structure.

The sorting method depends on the dimension of the state.² For $d = 1$, the particles are scalars and we set $H(x) = h(x) = x$ for $x \in [0, 1]$, where an ordinary sort suffices. For $d \geq 2$, the particles are ordered along a Hilbert space-filling curve. The Hilbert curve is a space-filling curve: a continuous surjective function $H : [0, 1] \rightarrow [0, 1]^d$ that fills the entire unit cube (Sagan, 1994). Its pseudo-inverse $h : [0, 1]^d \rightarrow [0, 1]$ maps each d -dimensional particle to a scalar key in $[0, 1]$. The key property of the Hilbert curve is that it preserves locality: points that are close in $[0, 1]^d$ are mapped by h to close keys in $[0, 1]$. Refer to Sagan (1994) and Butz (1969) for detailed discussion of the Hilbert curve and its properties; Hamilton and Rau-Chaplin (2008) and Chopin and Gerber (2015) for algorithms to compute the pseudo-inverse h for $d \geq 2$.

To sort the particles at time $t - 1$, each state is first mapped into the unit cube by a component-wise function $\psi : \mathcal{X} \rightarrow [0, 1]^d$, and the Hilbert key is then the pseudo-inverse h of the transformed state. A common choice for ψ is the logistic

²In `jax`, the ordering is computed with the compiled `jax.numpy.sort`; on the CPU this is a stable comparison sort, while on the GPU XLA sorts the packed integer Hilbert keys by radix sort, which is linear in N for a fixed key width, faster than $\mathcal{O}(N \log N)$ for higher dimensions.

transformation component-wise when $\mathcal{X} = \mathbb{R}^d$, which gives $\psi_i(x_i) = (1 + e^{-x_i})^{-1}$.

$$h_{t-1}^n = h \circ \psi(X_{t-1}^n) \in [0, 1]$$

The pseudo-inverse h preserves the low-discrepancy structure of the transformed point set $\{\psi(X_{t-1}^n)\}$: it suffices to take $\psi(\mathbf{x}) = (\psi_1(x_1), \dots, \psi_d(x_d))$ with each ψ_i a continuous and strictly monotone bijection of the real line onto $(0, 1)$, which maps a low-discrepancy point set into a low-discrepancy point set in the cube. The only requirement of SQMC is that the simulation of particle $\mathbf{x}_t^n$ from $\mathbf{x}_{t-1}^n$ can be written as a deterministic function of $\mathbf{x}_{t-1}^n$ and a fixed number of uniform variates.

Algorithm 3 describes the operations performed in SQMC. We always randomise the point sets (RQMC); this is the construction required for SQMC to provide an unbiased estimator of the normalising constant (Chopin and Gerber, 2015; Del Moral, 1996) and under the regularity conditions described in Section 2.1.2, its variance can be smaller than that of SMC. Details and further discussions on the algorithm are given in Chopin and Gerber (2015), Chopin and Gerber (2017) and Chopin and Papaspiliopoulos (2020).

Algorithm 3 SQMC

- 1: **At time 0,**
 - 2: Generate a QMC point set $u_0^{1:N}$ of dimension d .
 - 3: Compute $X_0^n = \Gamma_0(u_0^n)$ for $n = 1, \dots, N$.
 - 4: Compute $W_0^n = G_0(X_0^n) / \sum_{m=1}^N G_0(X_0^m)$ for $n = 1, \dots, N$.
 - 5: **for** $t = 1, \dots, T$ **do**
 - 6: Generate a QMC or RQMC point set $(u_t^{1:N}, v_t^{1:N})$ of dimension $d + 1$, where u_t^n is the first component and v_t^n the remaining d components of point n .
 - 7: **Hilbert sort:** find the permutation σ_t such that $h \circ \psi(X_{t-1}^{\sigma_t(1)}) \leq \dots \leq h \circ \psi(X_{t-1}^{\sigma_t(N)})$ if $d \geq 2$, or $X_{t-1}^{\sigma_t(1)} \leq \dots \leq X_{t-1}^{\sigma_t(N)}$ if $d = 1$.
 - 8: Generate $A_t^{1:N}$ by the inverse-CDF transform with inputs $\text{sort}(u_t^{1:N})$ and $W_{t-1}^{\sigma_t(1:N)}$, and compute $X_t^n = \Gamma_t(X_{t-1}^{\sigma_t(A_t^n)}, v_t^n)$ for $n = 1, \dots, N$.
 - 9: Compute $W_t^n = G_t(X_{t-1}^{\sigma_t(A_t^n)}, X_t^n) / \sum_{m=1}^N G_t(X_{t-1}^{\sigma_t(A_t^m)}, X_t^m)$ for $n = 1, \dots, N$.
 - 10: **end for**
-

2.1.4 Key Properties of SQMC

Computational Complexity. As noted above, the Hilbert sort makes the per-iteration cost of SQMC $\mathcal{O}(N \log N)$, compared with $\mathcal{O}(N)$ for SMC. The additional

cost comes from preserving the low-discrepancy structure of the particles through the Hilbert-sort.

Consistent Estimate. Under the same conditions as SMC, the SQMC estimator is *consistent*: the empirical filtering distribution converges to the true filtering distribution as $N \rightarrow \infty$ (Chopin and Gerber, 2015). The deterministic propagation through low-discrepancy points does not compromise the asymptotic correctness of the filter.

RQMC error. The error rate of SMC at iteration t is $C_t N^{-1/2}$, where C_t is a time-dependent constant. Most work focuses on the first factor C_t , either by establishing that the error is bounded uniformly in time (Del Moral, 2004; Whiteley, 2013), or by reducing C_t through more efficient algorithmic designs, such as better proposal kernels or resampling schemes (Del Moral et al., 2012). SQMC improves the second factor: its RQMC error is of the same order as the Monte Carlo error of SMC, $\mathcal{O}_P(N^{-1/2})$, but with a smaller constant (Chopin and Gerber, 2015). Under additional regularity conditions, the low-discrepancy structure can yield a faster rate.

Time-uniform error bounds. SQMC offers a stronger guarantee over time than SMC. Gerber (2026) suggests that a time-uniform almost-sure guarantee cannot exist for SMC: a fixed-size particle filter’s error exceeds any fixed threshold infinitely often. SQMC, by contrast, retains the time-uniform almost-sure guarantee: its error vanishes uniformly in time, $\lim_{N \rightarrow \infty} \sup_{t \geq 1} \|\hat{\eta}_t^N - \eta_t\| = 0$ almost surely. This makes SQMC attractive for long-horizon filtering problems.

2.2 Empirical evaluation: accelerator performance of the SQMC implementation

This section evaluates whether accelerator hardware improves the three operations that distinguish SQMC from a standard bootstrap particle filter: QMC point generation, Hilbert sorting of QMC-generated inputs, and SQMC filtering trajectories. All experiments compare against a shared implementation on CPU and GPU to isolate the compute advantage of parallel accelerators.

2.2.1 Experimental design and measurement

The experiments were conducted on an NVIDIA A100-SXM4 with 40 GB of device memory and 12 logical cores of an Intel Xeon CPU at 2.20 GHz. The hardware and software environment is recorded in Appendix C, Section “Software environment”.

2.2.2 QMC generation

Halton and Sobol’ sequences in Section 2.1.2 are both constructed by operations that are computed in integer arithmetic before a single conversion to floating point, which avoids floating-point cancellation. The Sobol’ generator is also solely computed by XOR operations, because each point’s coordinates are a function of its index alone so the whole batch can be generated independently in parallel. This makes both sequences well suited to implementation on GPU hardware. These results follow from (Keller et al., 2023). Another benefit of implementing the generators in `jax` is that all randomness is expressed through explicit key arrays: the key arrays are passed explicitly to the generator functions, which is deterministic and reproducible and requires no stateful Python state to be carried through compiled code (as compared to the stateful implementation in `scipy`). We consider dimensions and point counts

$$d \in \{2, 5, 10, 30, 60\},$$
$$N \in \{128, 256, 512, 2048, 8192, 32768\}.$$

Figure 2.3 reports the median runtime of the matched fresh-scramble timing configurations for the shared `jax` implementation on CPU and GPU, one panel per sequence.

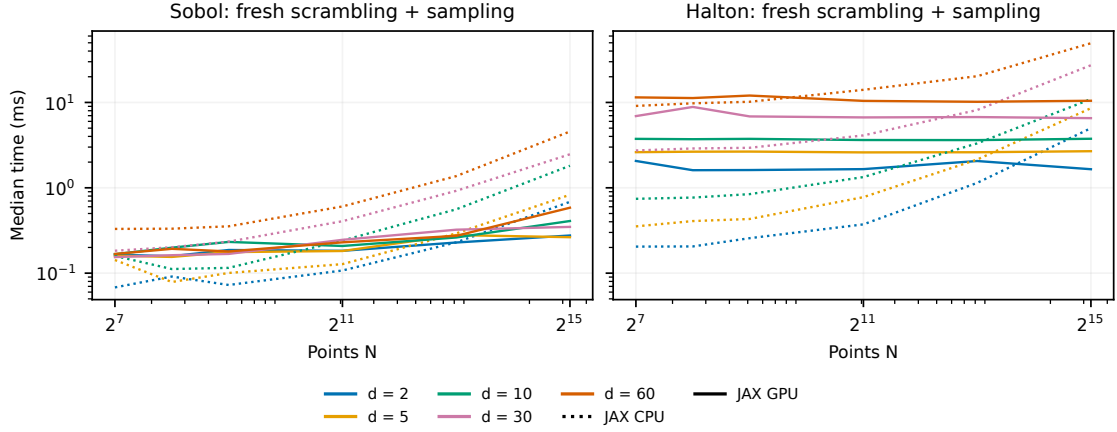


Figure 2.3: Median runtime in milliseconds for fresh scrambling and sampling of the Sobol’ (left) and Halton (right) sequences, for the shared `jax` implementation on CPU (dashed) and GPU (solid). Colours identify dimensions.

For Sobol’ generation, the first GPU advantage over CPU occurs at $N = 8192$ for $d = 2, 5$, at $N = 2048$ for $d = 10$, and at $N = 128$ for $d = 30, 60$. For Halton generation the GPU advantage appears only at larger counts: $N = 32768$ for $d = 2, 5, 10$, $N = 8192$ at $d = 30$, and $N = 2048$ at $d = 60$. The GPU is thus faster mainly at large point counts and higher dimensions, and small configurations often favour the CPU.

2.2.3 Hilbert sorting

Computing the Hilbert index of a point is a purely local, index-based operation that requires no lookup tables, so the deterministic index computations can be parallelised and results are exactly reproducible (Keller et al., 2022). The Hilbert-sorting experiment uses the same five dimensions as the QMC generation, with the point counts

$$N \in \{128, 256, 512, 2048, 8192, 32768\}.$$

Input generation is excluded from sorting time. Figure 2.4 shows the results for Sobol’-generated and Halton-generated inputs. The 60 comparisons compare the same shared `jax` sorter on CPU and GPU, using identical normal-transformed arrays within each pair.

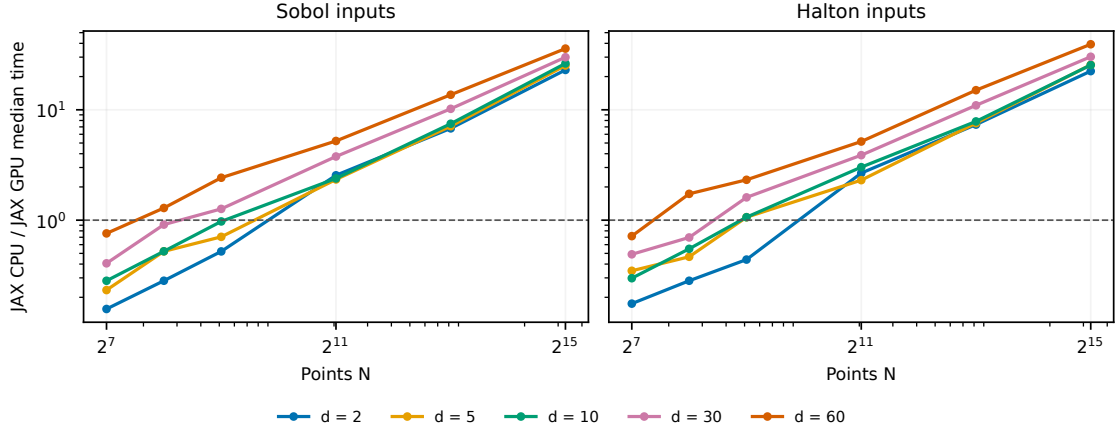


Figure 2.4: `jax` CPU/`jax` GPU median-runtime ratios for shared Hilbert sorting. Left: normal-transformed scrambled Sobol’ inputs; right: normal-transformed scrambled Halton inputs. Colours identify dimensions. Point generation and input transfer are excluded from sorting time; the horizontal line marks equal runtime. Points above the horizontal line favour the GPU.

GPU speedups are observed at higher dimensions and particle counts. At $N = 128$, GPU is slower than CPU in every tested dimension for both input families. For Sobol’ inputs, the first measured GPU advantage occurs at $N = 2048$ for $d = 2, 5, 10$, $N = 512$ for $d = 30$, and $N = 256$ for $d = 60$. For Halton inputs, the corresponding counts are 2048, 512, 512, 512, 256 in increasing dimension order. Thus the benefit of parallel sorting emerges only in higher dimensions and particle counts.

2.2.4 SQMC on CPU and GPU at a fixed iteration count

For SQMC, we use the linear Gaussian model

$$\begin{aligned} X_0 &\sim \mathcal{N}(0, I_d) \\ X_t | X_{t-1} &\sim \mathcal{N}(X_{t-1}, 0.25I_d) \\ Y_t | X_t &\sim \mathcal{N}(X_t, I_d) \end{aligned}$$

One observation dataset is generated for each dimension. The CPU and GPU use the same `jax` implementation and receive the same observations and matching replicate keys. The benchmark uses the shared SQMC filter builder, shared Hilbert ordering and freshly scrambled Sobol’ points of dimension $d + 1$ at each update, with separate initialisation and update randomisations. A timed trajectory includes initialisation and all 100 filtering observation updates, including scrambling, sorting, resampling, propagation, weighting, filtering-mean calculation and likelihood accumulation.

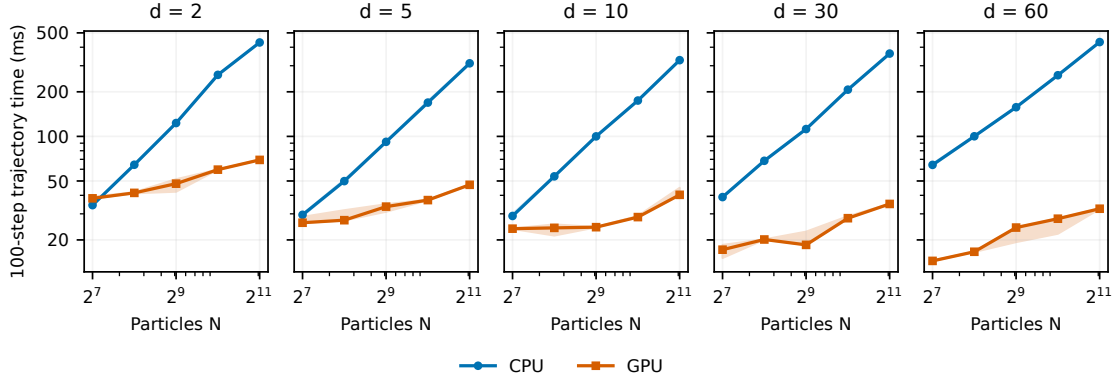


Figure 2.5: Median runtime of a complete 100-observation SQMC trajectory on CPU and GPU, at matched particle counts, with shaded bands showing the interquartile range. The same shared SQMC implementation is used in every panel.

We compare SQMC on GPU and CPU from two perspectives: how much faster the GPU executes the same filtering work, and what accuracy each backend attains within a fixed runtime budget. Figure 2.5 plots the runtime speedups $S_{d,N} = \tilde{\tau}_{\text{CPU},d,N} / \tilde{\tau}_{\text{GPU},d,N}$ over the 25 matched CPU/GPU pairs, where a value above one favours the GPU. At $N = 2048$, GPU speedups range from $6.2\times$ to $13.4\times$, and the CPU is faster only at $d = 2$, $N = 128$; at every other tested count the GPU is faster in all five dimensions.

Another measure compares how accurate each backend can be within a fixed time budget. The error of the filtering mean estimate is summarised by the normalised RMSE,

$$E_{d,N} = \left[\frac{1}{RTd} \sum_{r=1}^R \sum_{t=1}^T \sum_{j=1}^d \frac{(\hat{\mu}_{tj}^{(r)} - \mu_{tj})^2}{P_{tj}} \right]^{1/2}, \quad (2.2)$$

and Figure 2.6 reports this RMSE for the configuration each backend can afford under the tested median-runtime budgets. Within those budgets, the GPU attains a lower normalised filtering RMSE than the CPU in every dimension, so it reaches a given accuracy in less time. Together, the two figures show that the GPU is faster for a fixed amount of filtering work and more accurate under a fixed time budget.

All code is written in the public repository `ryantjx/sqmc-cuthbert`, and the SQMC implementation is being integrated into `state-space-models/cuthbert` under issue #255.

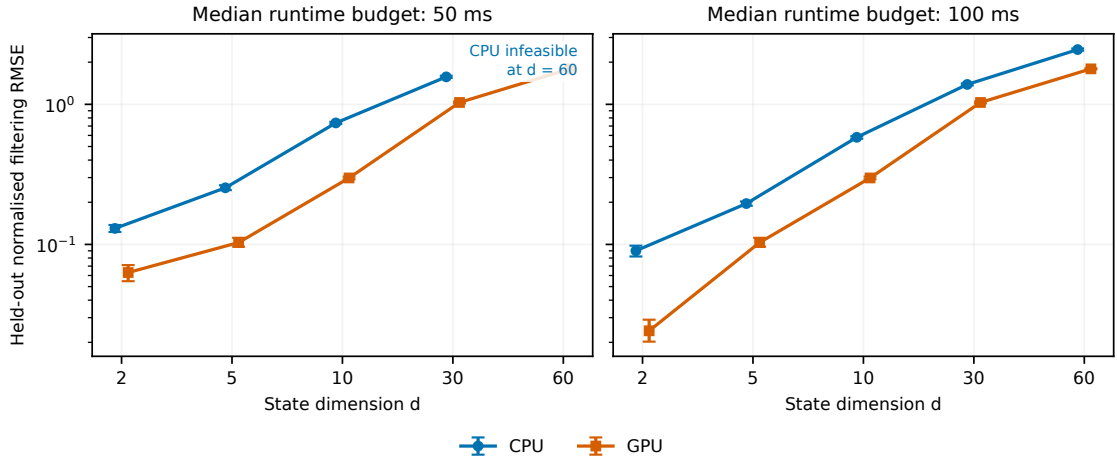


Figure 2.6: Filtering RMSE of the SQMC configuration selected under each median-runtime budget, evaluated on the same sixteen randomisations used for selection. Error bars are 95% bootstrap intervals from sixteen randomisations and 500 resamples, conditional on one dataset per dimension; they represent Monte Carlo randomisation variability only, not variability across observation datasets or hardware sessions. CPU has no eligible configuration at $d = 60$ under 50 ms. Neither backend has an eligible configuration under 10 ms.

Chapter 3

Rao-Blackwellised SQMC for Football Matches

In this chapter, we develop a correlated Gaussian state-space model for football match outcomes. The observation likelihood depends on only the two playing teams, so the effective dimension is far smaller than the $2K$ -dimensional latent state. We exploit this sparsity through Rao-Blackwellisation, integrating out the non-playing teams analytically and running SQMC on the four playing-team coordinates, overcoming the high-dimensional limitations of SQMC.

3.1 Background

Football is a high-dimensional problem with sparse pairwise observations. Let each team's latent state be a two-dimensional vector of attacking and defensive strengths, $x_t^k = (x_t^{k,\text{attack}}, x_t^{k,\text{defence}})$ for $k \in \{1, 2, \dots, K\}$. Then, the latent state of all K teams is $x_t \in \mathbb{R}^{2K}$. However, each match involves only the latent states of the two playing teams at time t , $x_t^{\mathcal{O}_t} = (x_t^{(i_t)}, x_t^{(j_t)}) \in \mathbb{R}^4$, through the local observation model $G_t(y_t | x_t^{\mathcal{O}_t})$, which represents the outcome of a single football match. The general state-space model notation introduced in Section 2.1 is inefficient for this problem: it represents the full joint distribution over all teams, even though each likelihood contribution depends on only four coordinates.

The factorial hidden Markov model (fHMM) was introduced by Ghahramani and Jordan (Ghahramani and Jordan, 1995, 1997) to tackle problems with structures that can be factorised into individual components. A factorial state-space model (fSSM) is the corresponding state-space construction, extending from the general

state-space model defined in Equation (2.1). Its state x is naturally partitioned into a collection of components k , each of which evolves independently of one another. The observation likelihood G_t may depend jointly on a single component or on a collection of components. Let $\mathcal{O}_t \subseteq \{1, \dots, K\}$ denote the subset of components involved in the observation at time t , and write $x_t^{\mathcal{O}_t} = (x_t^k)_{k \in \mathcal{O}_t}$ for the corresponding collection of components.

The fSSM model makes three assumptions: (1) the initial latent states are a priori independent (independent given any observations); (2) the evolution m_t of each component k is independent of the other components; (3) given the latent states $x_t^{\mathcal{O}_t}$, the observations y_t are conditionally independent of all other components $-\mathcal{O}_t$. Applying these to the general joint density of Equation (2.1), the initial distribution factorises as $\prod_{k=1}^K m_0(x_0^k)$, the transition factor as $\prod_{k=1}^K \prod_{t=1}^T m_t(x_t^k | x_{t-1}^k)$, and the observation term depends only on the playing pair, $\prod_{t=1}^T G_t(y_t | x_t^{\mathcal{O}_t})$. The joint density of the fSSM therefore factorises as

$$p(x_{0:T}^{1:K}, y_{1:T}) = \left[\prod_{k=1}^K m_0(x_0^k) \prod_{t=1}^T m_t(x_t^k | x_{t-1}^k) \right] \prod_{t=1}^T G_t(y_t | x_t^{\mathcal{O}_t}). \quad (3.1)$$

This reduces the computational complexity of the joint distribution from $\mathcal{O}(N \cdot K)$ to $\mathcal{O}(N + K)$ terms, which is a significant improvement for large K .

A recent paper by Duffield et al. (2024) utilized the fSSM structure and demonstrated its effectiveness in inference tasks for online skill rating and sports. Components can represent either an individual or a team's skill level. Inference then maintains a factorial filtering distribution, which factorises as a product of per-component marginals,

$$p(x_t | y_{1:t}) = \prod_{k=1}^K p(x_t^k | y_{1:t}). \quad (3.2)$$

Since the observation at time t involves only the pair $\mathcal{O}_t = \{i_t, j_t\}$, the prediction step propagates each component independently,

$$p(x_t^k | y_{1:t-1}) = \int m_t(x_t^k | x_{t-1}^k) p(x_{t-1}^k | y_{1:t-1}) dx_{t-1}^k, \quad (3.3)$$

and the update step assimilates the match result in the joint distribution of the playing pair,

$$p(x_t^{\mathcal{O}_t} | y_{1:t}) \propto G_t(y_t | x_t^{\mathcal{O}_t}) \prod_{k \in \mathcal{O}_t} p(x_t^k | y_{1:t-1}). \quad (3.4)$$

For each component $k \in \mathcal{O}_t$, the marginal filtering distribution is then recovered by integrating the joint update over the remaining components of the playing pair,

$$p(x_t^k | y_{1:t}) = \int p(x_t^{\mathcal{O}_t} | y_{1:t}) dx_t^{\mathcal{O}_t \setminus \{k\}}, \quad k \in \mathcal{O}_t, \quad (3.5)$$

and we obtain the filtering distribution in the factorial form of Equation (3.2). Figure 3.1 contrasts the conditional independence structure of a general state-space model with that of a fSSM: in the latter, each component evolves independently and the observation at time t couples only the components in $\mathcal{O}_t$. The main approximation is the repeated projection of the coupled posterior back to a product of component-level marginals, which discards the cross-component covariance after every match.

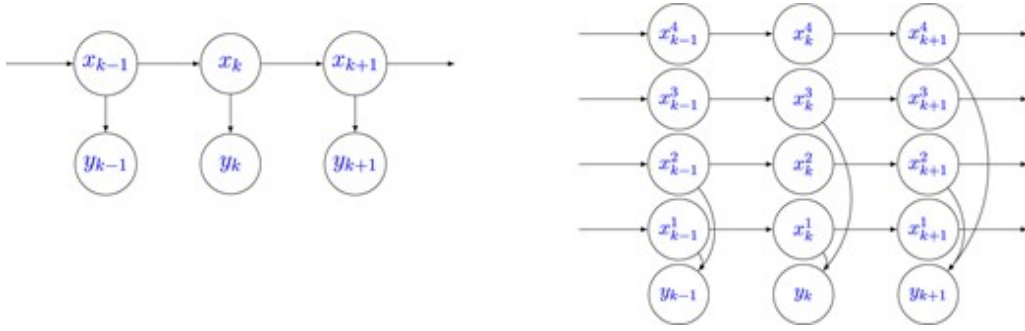


Figure 3.1: Conditional independence structure of a general state-space model (left) and of a factorial state-space model with a pairwise observation model (right), reproduced from Duffield et al. (2024). In the fSSM, each component evolves independently and the observation at time t couples only the components in $\mathcal{O}_t$.

Although efficient, fSSM is a restrictive model for football. Shared players, clubs, leagues and tournaments are potential sources of dependence between national teams that are not represented by a model whose team processes are *a priori* independent (i.e. assumed to be independent without any observations). Furthermore, match outcomes create dependencies that inform us on the strengths and uncertainty of other teams (e.g. if team A beats team B , and team B beats team C , we can infer that team A is likely stronger than team C). This information is discarded in the fSSM by approximating the filtering distribution as a product of independent team marginals as defined in Equation (3.2).

The model introduced in this chapter uses the original state-space model of Equation (2.1). Since the full latent state is high-dimensional, a direct particle filter costs $\mathcal{O}(T \cdot N \cdot K)$ per run, where N is the number of particles and T the number of observations. To avoid this, we restrict the transition to linear-Gaussian and apply Rao-Blackwellisation: we sample the four playing-team coordinates of each particle and integrate the non-playing teams out analytically, so that their conditional mean and covariance are updated in closed form by the Kalman filter. This reduces the

propagation to a four-dimensional block, and SQMC is then applied to exactly this block: at each match we draw a RQMC point set for the four playing-team coordinates, order the particles along a Hilbert curve and resample, so that the low-discrepancy structure is preserved. The per-observation cost is $\mathcal{O}(4N)$, independent of K , and because SQMC now propagates four coordinates rather than $2K$, it can exploit the low-discrepancy properties of quasi-Monte Carlo despite the high nominal state dimension. Section 3.5 develops this construction in detail.

3.2 Model

We represent the latent state of each team as a two-dimensional vector $x = (x^{\text{attack}}, x^{\text{defence}})$, so the full latent state of K teams is $x \in \mathbb{R}^{2K}$, as introduced in Section 3.1. A larger attack value represents a stronger attack; likewise a larger defense value represents a stronger defense component. Our initial distribution of latent states is given by

$$x_0 \in \mathbb{R}^{2K}, \quad x_0 \sim \mathcal{N}_{2K}(\mu_0, \Sigma_0) \quad (3.6)$$

We make the further assumption that the relationship between attack and defence of all teams is the same, represented by the matrix $B \in \mathbb{R}^{2 \times 2}$. This assumption allows us to simplify the covariance matrix into a Kronecker product of the form $\Sigma_0 = \Gamma_0 \otimes B$, where $\Gamma_0 \in \mathbb{R}^{K \times K}$ is the prior filter covariance that represents the relationship between teams. We require the conditions $\Gamma_0 \succ 0$, $B \succ 0$ and $\det(B) = 1$. The condition $\det(B) = 1$ ensures that the scale of the Kronecker product is identifiable since $(c\Gamma_0) \otimes (B/c) = \Gamma_0 \otimes B$ (i.e. we fix the determinant of B so that Γ_0 is the free parameter in the Kronecker product). This parameterization is a natural choice for football as we can expect that attack and defence strengths of teams across all teams should share a similar relationship. It also reduces the number of free parameters in the initial covariance matrix from $K(2K + 1)$ to $K(K + 1)/2 + 2$ as seen in Proposition A.1, allowing us to efficiently compute the models in the filtering, smoothing and parameter estimation steps. We model the evolution of latent states as an Ornstein-Uhlenbeck process, a mean-reverting stochastic process, given by

$$x_t \mid x_{t-1} \sim \mathcal{N}(\mu_0 + \phi_t(x_{t-1} - \mu_0), Q_t) \quad (3.7)$$

where we define $\phi_t = \exp(-\kappa \Delta t_t)$, $\Delta t_t = t_t - t_{t-1}$ and $Q_t = (1 - \phi_t^2)\Sigma_0$ is the

covariance matrix of the process noise. κ is the mean-reversion rate and Δt_t is the time difference between t and $t - 1$, measured in days. This behaviour is desirable in football as we expect average skill levels to revert to a long-run marginal distribution of $\mathcal{N}(\mu_0, \Sigma_0)$ over time without any new observations.

Each match only involves two teams, so we represent the latent states of the two playing teams as $x_t^{\mathcal{O}_t} = (x_t^{(i_t)}, x_t^{(j_t)})$. Our observation is a bivariate vector of scores for team i_t and team j_t , represented as $y_t = (y_t^{(i_t)}, y_t^{(j_t)})$. The likelihood function follows the bivariate Poisson distribution that Karlis and Ntzoufras (2003) proposed for sports data.

$$G_t(y_t \mid x_t^{\mathcal{O}_t}) = e^{-(\lambda_1 + \lambda_2 + \lambda_3)} \frac{\lambda_1^{y_t^{(i_t)}}}{y_t^{(i_t)}!} \frac{\lambda_2^{y_t^{(j_t)}}}{y_t^{(j_t)}!} \sum_{k=0}^{\min(y_t^{(i_t)}, y_t^{(j_t)})} \binom{y_t^{(i_t)}}{k} \binom{y_t^{(j_t)}}{k} k! \left(\frac{\lambda_3}{\lambda_1 \lambda_2} \right)^k \quad (3.8)$$

where $\lambda_1 = \exp(\alpha + x_t^{\text{att}, i_t} - x_t^{\text{def}, j_t})$, $\lambda_2 = \exp(\alpha + x_t^{\text{att}, j_t} - x_t^{\text{def}, i_t})$, $\lambda_3 = \exp(\beta)$. The parameter α and β are static parameters of the model. α can be used to model home-team advantage by estimating α^h in λ_1 and α^a in λ_2 .

3.3 Rao-Blackwellised SMC (RB-SMC)

In our previous Section 3.2, we state that only the latent states of the two playing teams $x_t^{\mathcal{O}_t}$ are relevant to the likelihood function G_t . Defining the time-varying non-playing latent states as $x_t^{\mathcal{R}_t} = x_t^{-\mathcal{O}_t}$, the Rao-Blackwellised SMC (RB-SMC) algorithm therefore targets the marginal posterior of the coordinates of the playing teams $p(x_{0:T}^{\mathcal{O}} \mid y_{1:T})$ and recovers the non-playing teams $p(x_{0:T}^{\mathcal{R}} \mid x_{0:T}^{\mathcal{O}}, y_{1:T})$ in closed form. This is similar to the Rao-Blackwellised particle filter construction of Doucet et al. (2000). We can then factorize the full joint posterior,

$$p(x_{0:T} \mid y_{1:T}) = p(x_{0:T}^{\mathcal{O}} \mid y_{1:T}) \cdot p(x_{0:T}^{\mathcal{R}} \mid x_{0:T}^{\mathcal{O}}, y_{1:T}) \quad (3.9)$$

Since the likelihood depends only on the sampled coordinates at each time, the second factor simplifies to

$$p(x_t^{\mathcal{R}_t} \mid x_{1:t}^{\mathcal{O}_{1:t}}, y_{1:t}) = p(x_t^{\mathcal{R}_t} \mid x_{1:t}^{\mathcal{O}_{1:t}}).$$

As stated before, our transition model is linear-Gaussian, so the covariance recursions are deterministic and do not depend on the sampled coordinates or the particle paths; hence Σ_t is shared across all particles while the means $\mu_t^{(i)}$ carries the particle-specific information. The RB-SMC algorithm has the same steps as the SMC algorithm as defined in Algorithm 2, the only difference is that we only sample the latent state of playing teams $x_t^{\mathcal{O}_t}$ at each time step and compute the marginal posterior for the non-playing teams in closed form. The RB-SMC algorithm is as follows:

1 Prediction: Compute the predictive mean and covariance matrix for the full state:

$$\mu_{t|t-1}^{(i)} = \mu_0 + \phi_t(\mu_{t-1}^{(i)} - \mu_0)$$

$$\Sigma_{t|t-1} = \phi_t^2 \Sigma_{t-1} + (1 - \phi_t^2) \Sigma_0$$

2 Bootstrap Proposal Sampling: For particle i , the full predictive distribution is

$$x_t \mid I_{t-1} = i, y_{1:t-1} \sim \mathcal{N}(\mu_{t|t-1}^{(i)}, \Sigma_{t|t-1}) \quad (3.10)$$

At each observation, only the two playing teams are sampled. We extract the predictive mean and covariance for the playing teams' coordinates $x_t^{\mathcal{O}_t}$, and the following predictive marginal under the bootstrap proposal is defined as,

$$q_t^{(i)}(x_t^{\mathcal{O}_t}) = p(x_t^{\mathcal{O}_t} \mid I_{t-1} = i, y_{1:t-1}) = \mathcal{N}(\mu_{t|t-1}^{\mathcal{O}_t, (i)}, \Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t})$$

3 Computing log-weights: For the propagate-weight-resample ordering, compute the unnormalised log-weights for each particle i using the previous weight and the likelihood function G_t ,

$$\log \tilde{w}_t^{(i)} = \log w_{t-1}^{(i)} + \log G_t(y_t \mid x_t^{\mathcal{O}_t, (i)})$$

and normalise,

$$w_t^{(i)} = \frac{\tilde{w}_t^{(i)}}{\sum_{n=1}^N \tilde{w}_t^{(n)}}$$

4 Exact Marginalisation of Non-Playing Teams: Given the predictive joint distribution and conditioning on the playing teams' coordinates $x_t^{\mathcal{O}_t}$, the non-playing

coordinates $x_t^{\mathcal{R}_t}$ are Gaussian with a closed-form mean and covariance matrix defined by the Kalman filter equations,

$$x_{t|t-1}^{\mathcal{R}_t, (i)} \mid x_t^{\mathcal{O}_t, (i)} \sim \mathcal{N} \left(\mu_{t|t-1}^{\mathcal{R}|\mathcal{O}, (i)}, \Sigma_{t|t-1}^{\mathcal{R}|\mathcal{O}} \right)$$

$$\begin{aligned} \mu_{t|t-1}^{\mathcal{R}_t|\mathcal{O}_t, (i)} &= \mu_{t|t-1}^{\mathcal{R}_t} + K_t \left(x_t^{\mathcal{O}_t, (i)} - \mu_{t|t-1}^{\mathcal{O}_t} \right) \\ \Sigma_{t|t-1}^{\mathcal{R}_t|\mathcal{O}_t} &= \Sigma_{t|t-1}^{\mathcal{R}_t \mathcal{R}_t} - K_t \Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{R}_t} \\ K_t &= \Sigma_{t|t-1}^{\mathcal{R}_t \mathcal{O}_t} \left(\Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} \right)^{-1} \end{aligned}$$

In Section 2.1.1, we restrict out our initial covariance to the form of a Kronecker product $\Sigma_0 = \Gamma_0 \otimes B$. Since our transition model is linear-Gaussian, the Kronecker structure is preserved recursively by induction,

$$\begin{aligned} \Sigma_{t|t-1} &= \phi_t^2 \Sigma_{t-1} + (1 - \phi_t^2) \Sigma_0 \\ &= \phi_t^2 (\Gamma_{t-1} \otimes B) + (1 - \phi_t^2) (\Gamma_0 \otimes B) \\ &= \left(\phi_t^2 \Gamma_{t-1} + (1 - \phi_t^2) \Gamma_0 \right) \otimes B \\ &= \Gamma_{t|t-1} \otimes B \end{aligned}$$

so that the covariance blocks for the playing and non-playing teams factor as:

$$\Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} = \Gamma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} \otimes B, \quad \Sigma_{t|t-1}^{\mathcal{R}_t \mathcal{R}_t} = \Gamma_{t|t-1}^{\mathcal{R}_t \mathcal{R}_t} \otimes B, \quad \Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{R}_t} = \Gamma_{t|t-1}^{\mathcal{O}_t \mathcal{R}_t} \otimes B$$

where $\Gamma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} \in \mathbb{R}^{2 \times 2}$ is the submatrix of $\Gamma_{t|t-1}$ for the two playing teams and $\Gamma_{t|t-1}^{\mathcal{R}_t \mathcal{R}_t} \in \mathbb{R}^{(K-2) \times (K-2)}$ is the submatrix for the non-playing teams. Using the mixed-product property $(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$ and the inverse property $(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}$, the Kalman gain factorises,

$$\begin{aligned}
K_t &= \Sigma_{t|t-1}^{\mathcal{R}_t \mathcal{O}_t} \left(\Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} \right)^{-1} \\
&= \left(\Gamma_{t|t-1}^{\mathcal{R}_t \mathcal{O}_t} \otimes B \right) \left(\Gamma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} \otimes B \right)^{-1} \\
&= \left(\Gamma_{t|t-1}^{\mathcal{R}_t \mathcal{O}_t} \otimes B \right) \left(\left(\Gamma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} \right)^{-1} \otimes B^{-1} \right) \\
&= \left(\Gamma_{t|t-1}^{\mathcal{R}_t \mathcal{O}_t} \left(\Gamma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} \right)^{-1} \right) \otimes \left(B B^{-1} \right) \\
&= \tilde{K}_t \otimes I_2
\end{aligned}$$

$$\tilde{K}_t = \Gamma_{t|t-1}^{\mathcal{R}_t \mathcal{O}_t} \left(\Gamma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} \right)^{-1}$$

and the conditional covariance of the non-playing teams becomes

$$\Sigma_{t|t-1}^{\mathcal{R}_t | \mathcal{O}_t} = \left(\Gamma_{t|t-1}^{\mathcal{R}_t \mathcal{R}_t} - \tilde{K}_t \Gamma_{t|t-1}^{\mathcal{O}_t \mathcal{R}_t} \right) \otimes B$$

Two distinct Kalman gains appear in the recursion. The gain $\tilde{K}_t \in \mathbb{R}^{(K-2) \times 2}$ conditions only the *non-playing* teams on the playing teams and is used for the conditional covariance $\Sigma_{t|t-1}^{\mathcal{R}_t | \mathcal{O}_t}$ (equivalently, in the full state space, $K_t = \tilde{K}_t \otimes I_2$). To update the *full component mean* and the *between-team* covariance, define the full team-space gain

$$\bar{K}_t = \Gamma_{t|t-1}^{\mathcal{O}_t} \left(\Gamma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} \right)^{-1} \in \mathbb{R}^{K \times 2},$$

where $\Gamma_{t|t-1}^{\mathcal{O}_t}$ contains all rows and the two playing-team columns of $\Gamma_{t|t-1}$. The two gains are the same conditioning operator on different rows: the $\mathcal{O}_t$ rows of $\bar{K}_t$ form the identity, and its $\mathcal{R}_t$ rows coincide with $\tilde{K}_t$. The complete component mean is updated as

$$\mu_t^{(i)} = \mu_{t|t-1}^{(i)} + (\bar{K}_t \otimes I_2) \left(x_t^{\mathcal{O}_t, (i)} - \mu_{t|t-1}^{\mathcal{O}_t, (i)} \right)$$

The rows of $\bar{K}_t$ belonging to $\mathcal{O}_t$ form the identity, so the playing-team entries of $\mu_t^{(i)}$ equal the sampled entries $x_t^{\mathcal{O}_t, (i)}$, while the $\mathcal{R}_t$ rows recover the conditional mean of the non-playing teams in closed form. The team-space filtered covariance Γ_t is defined by the Schur complement

$$\Gamma_t = \Gamma_{t|t-1} - \bar{K}_t \Gamma_{t|t-1}^{\mathcal{O}_t} = \Gamma_{t|t-1} - \Gamma_{t|t-1}^{\mathcal{O}_t} \left(\Gamma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} \right)^{-1} \Gamma_{t|t-1}^{\mathcal{O}_t}$$

and hence to recover the full filtered covariance Σ_t , one can simply compute the Kronecker product of Γ_t with B . The full filtered covariance is sparse because the components of the playing teams of Γ_t are zero.

$$\Sigma_t = \Gamma_t \otimes B = \begin{pmatrix} \mathbf{0}_{4 \times 4} & \mathbf{0}_{4 \times 2(K-2)} \\ \mathbf{0}_{2(K-2) \times 4} & \Gamma_t^{\mathcal{R}_t \mathcal{R}_t} \otimes B \end{pmatrix}$$

where $\mathbf{0}_{4 \times 4}$ is the zero block for the four coordinates of the two playing teams, whose latent states are now known (sampled) values with no remaining uncertainty, and $\Gamma_t^{\mathcal{R}_t \mathcal{R}_t} \otimes B$ is the non-zero conditional covariance of the non-playing teams. Intuitively, once a team has played a match, its skill level is exactly known at that point in time. Additionally, as mentioned in Section 3.2, the Kronecker structure reduces the number of free parameters in the covariance matrix from $K(2K+1)$ to $K(K+1)/2 + 2$. The computation of Γ_t is reduced from $\mathcal{O}((2K)^3)$ to $\mathcal{O}(K^3)$ and we avoid any particle-specific covariance recursions.

5 Resampling: Resample the particles according to the normalised weights $\{w_t^{(i)}\}_{i=1}^N$ and set the weights to $w_t^{(i)} = 1/N$ for all particles.

3.4 RB-SMC Smoothing

In standard particle-based Forward Filtering Backward Simulation (FFBSi), the backward state at time t is sampled from the discrete set of filtering particles, with probabilities determined by their filtering weights and compatibility with the sampled state at time $t+1$ (Godsill et al., 2004). In the RB-SMC formulation, discrete point-particle FFBSi cannot be applied directly, because each filtering particle represents a Gaussian component rather than a complete state. Using the fact that our transition is linear and Gaussian, our terminal posterior filtering distribution can be approximated as a weighted Gaussian mixture with a shared covariance matrix Σ_t across all particles.

$$p(x_T \mid y_{1:T}) \approx \sum_{i=1}^N w_T^{(i)} \mathcal{N}(\mu_T^{(i)}, \Sigma_T) \quad (3.11)$$

This means that a one-step backward kernel is available in closed form to obtain a smoothed trajectory: first select a Gaussian component using its backward weight, and then draw the full state from that component's conditional Gaussian distribution.

3.4.1 Deriving the one-step backward kernel

The one-step backward kernel satisfies

$$p(x_t \mid x_{t+1}, y_{1:t}) \propto p(x_t \mid y_{1:t}) p(x_{t+1} \mid x_t). \quad (3.12)$$

To derive this kernel, apply Bayes' rule to the filtering mixture at time t ,

$$p(x_t \mid y_{1:t}) \approx \sum_{i=1}^N w_t^{(i)} \mathcal{N}(x_t \mid \mu_t^{(i)}, \Sigma_t), \quad (3.13)$$

and the linear-Gaussian transition $p(x_{t+1} \mid x_t) = \mathcal{N}(x_{t+1} \mid a_{t+1}(x_t), Q_{t+1})$, where $a_{t+1}(x_t) = \mu_0 + \phi_{t+1}(x_t - \mu_0)$ and $Q_{t+1} = (1 - \phi_{t+1}^2)\Sigma_0$. Substituting the mixture and the transition, the backward kernel is proportional to the product

$$p(x_t \mid x_{t+1}, y_{1:t}) \propto \sum_{i=1}^N w_t^{(i)} \mathcal{N}(x_t \mid \mu_t^{(i)}, \Sigma_t) \mathcal{N}(x_{t+1} \mid a_{t+1}(x_t), Q_{t+1}),$$

in which the transition density depends on x_t through $a_{t+1}(x_t)$. Completing the square in x_t for each component, the product factorises into the predictive density of x_{t+1} under component i , $\mathcal{N}(x_{t+1} \mid a_{t+1}^{(i)}, R_{t+1})$, where $a_{t+1}^{(i)} = \mu_0 + \phi_{t+1}(\mu_t^{(i)} - \mu_0)$ and $R_{t+1} = \phi_{t+1}^2 \Sigma_t + (1 - \phi_{t+1}^2)\Sigma_0$, times a conditional Gaussian density in x_t ; the first factor acts as a component weight, so the backward kernel is itself a Gaussian mixture,

$$p(x_t \mid x_{t+1}, y_{1:t}) \approx \sum_{i=1}^N w_{t|t+1}^{(i)} \mathcal{N}(x_t \mid \mu_{t|t+1}^{(i)}, \Sigma_{t|t+1}), \quad (3.14)$$

with backward weights

$$w_{t|t+1}^{(i)} \propto w_t^{(i)} \mathcal{N}(x_{t+1} \mid a_{t+1}^{(i)}, R_{t+1}), \quad (3.15)$$

and, by the Rauch-Tung-Striebel (RTS) equations, the corrected mean and covariance of each component

$$\mu_{t|t+1}^{(i)} = \mu_t^{(i)} + J_t (x_{t+1} - a_{t+1}^{(i)}), \quad \Sigma_{t|t+1} = \Sigma_t - J_t R_{t+1} J_t^\top, \quad J_t = \Sigma_t \phi_{t+1} R_{t+1}^{-1}. \quad (3.16)$$

Sampling from this mixture therefore proceeds in two steps: draw the component index $I_t \sim \text{Categorical}(w_{t|t+1}^{1:N})$, then draw the full state $x_t \sim \mathcal{N}(\mu_{t|t+1}^{(I_t)}, \Sigma_{t|t+1})$. Repeating this procedure from $t = T - 1$ down to $t = 0$ produces a single complete smoothed trajectory $\{X_0^*, \dots, X_T^*\}$.

3.4.2 The RB-SMC smoothing algorithm

The RB-SMC smoothing algorithm is as follows:

1 Initialisation ($t = T$): Sample the terminal particle index I_T using the normalised weights $w_T^{(i)}$, and draw the full terminal state from the corresponding Gaussian mixture,

$$I_T \sim \text{Categorical}(w_T^{1:N}), \quad X_T^* \sim \mathcal{N}(\mu_T^{(I_T)}, \Sigma_T)$$

2 Backward recursion (for $t = T - 1, T - 2, \dots, 0$): Given the already-sampled state X_{t+1}^* , the backward kernel derived in Section 3.4 is a Gaussian mixture, and the recursion proceeds in three sub-steps.

2a Compute the backward weights: For every particle i , propagate the filtering mean through the OU dynamics and form the predictive covariance,

$$a_{t+1}^{(i)} = \mu_0 + \phi_{t+1}(\mu_t^{(i)} - \mu_0), \quad R_{t+1} = \phi_{t+1}^2 \Sigma_t + (1 - \phi_{t+1}^2) \Sigma_0$$

then weight each component by its filtering weight and the predictive density of the conditioning state X_{t+1}^* ,

$$w_{t|t+1}^{(i)} \propto w_t^{(i)} \mathcal{N}(X_{t+1}^* | a_{t+1}^{(i)}, R_{t+1})$$

normalised so that $\sum_{i=1}^N w_{t|t+1}^{(i)} = 1$.

2b Select and correct the backward component: Sample the particle index from the backward weights,

$$I_t \sim \text{Categorical}(w_{t|t+1}^{1:N})$$

and condition the chosen component on X_{t+1}^* through the RTS equations with the gain

$$J_t = \Sigma_t \phi_{t+1} R_{t+1}^{-1}$$

$$\begin{aligned} \mu_{t|t+1}^{(I_t)} &= \mu_t^{(I_t)} + J_t (X_{t+1}^* - a_{t+1}^{(I_t)}) \\ \Sigma_{t|t+1} &= \Sigma_t - J_t R_{t+1} J_t^\top \end{aligned}$$

2c Draw the smoothed state: Sample the full state from the corrected component,

$$X_t^* \sim \mathcal{N}(\mu_{t|t+1}^{(I_t)}, \Sigma_{t|t+1})$$

The recursion terminates at $t = 0$, yielding the smoothed trajectory $\{X_0^*, \dots, X_T^*\}$. Repeating this procedure M times produces M smoothed trajectories $\{X_{0:T}^{*(m)}\}_{m=1}^M$ which is used to obtain the smoothing distribution $p(x_0 \mid y_{1:T})$. The M trajectories can be parallelised across M , while each trajectory retains a sequential backward recursion over T steps, each requiring $\mathcal{O}(N)$ mixture evaluations.

The resulting algorithm is similar to the pure particle smoother of Godsill et al. (2004), but the backward weights combine the forward filtering weights with the predictive density, while the RTS backward estimates determine the Gaussian mixture component from which the full state is drawn (Särkkä et al., 2012). This backward-simulation construction for a Rao-Blackwellised particle filter on a conditionally linear-Gaussian model follows Lindsten et al. (2012), and the use of backward simulation in particle methods more generally is discussed by Lindsten and Schön (2011).

3.5 Rao-Blackwellised SQMC (RB-SQMC)

Since the likelihood function depends only on the latent states of the two playing teams, our 'effective dimension' of the model is $d = 2 \times 2 = 4$ instead of the full state dimension of $2K$. This motivates the use of SQMC methods, which are known to perform better than SMC methods in low-dimensional settings as seen in Section 2.1.4.

Chopin and Gerber (2017) show that SQMC methods can be applied to high-dimensional state-space models if the transition and likelihood functions only depend on a subset of the state coordinates. In our case, we Hilbert-sort the $2K$ -dimensional particles using the 4-dimensional predicted means of the two playing teams, $\Lambda_t(\mu_{t|t-1}^{\mathcal{O}_t(i)})$, with the assumption that the 4-dimensional predicted means capture the particle diversity in the coordinates that determine the current likelihood. The likelihood is local because $G_t(x_{t-1}, x_t) = G_t(x_t^{\mathcal{O}_t})$, so the potential function depends only on the propagated coordinates of the two playing teams. These two characteristics mean that the locality condition of the cited argument is satisfied at the current step.

Similarly in Section 3.3, the RB-SQMC is the same as SQMC algorithm defined in Algorithm 3, with the only changes being made in the indexing of the teams being played $x_t^{\mathcal{O}}$. Our algorithm, the RB-SQMC, follows these steps:

1 Generate N QMC points: Generate N RQMC points $(u_t^{1:N}, v_t^{1:N})$ of dimension $1 + (2 \times 2) = 5$, where u_t is the first component, used for ancestor selection, and v_t

is the vector of $d = 4$ components used to propagate the latent states of the two playing teams, x_t^i and x_t^j .

2 Hilbert Sort: Apply the deterministic OU prediction to each Rao-Blackwellised component. The predictive mean and covariance of the full state are

$$\mu_{t|t-1}^{(i)} = \mu_0 + \phi_t \left(\mu_{t-1}^{(i)} - \mu_0 \right), \quad \Sigma_{t|t-1} = \phi_t^2 \Sigma_{t-1} + (1 - \phi_t^2) \Sigma_0$$

so that the playing-team block $\mu_{t|t-1}^{\mathcal{O}_t, (i)}$ and $\Sigma_{t|t-1}^{\mathcal{O}_t, \mathcal{O}_t}$ are the corresponding sub-blocks. Define the four-dimensional sorting key as $z_t^{(i)} = \text{vec} \left(\mu_{t|t-1}^{\mathcal{O}_t, (i)} \right) \in \mathbb{R}^4$. Let $\psi_t : \mathbb{R}^4 \rightarrow [0, 1]^4$ standardise and map these coordinates to the unit hypercube, and let $h : [0, 1]^4 \rightarrow [0, 1]$ denote the inverse Hilbert-curve mapping. Find a permutation σ_t such that

$$h \left(\psi_t(z_t^{(\sigma_t(1))}) \right) \leq \dots \leq h \left(\psi_t(z_t^{(\sigma_t(N))}) \right)$$

Use the previous normalised particle weights in the same order,

$$\left(W_{t-1}^{(\sigma_t(1))}, \dots, W_{t-1}^{(\sigma_t(N))} \right)$$

Once an ancestor is selected, its complete Rao-Blackwellised component, including the means of the non-playing teams, is retained and propagated.

3 Resampling by inverse transform: Sort the complete QMC pairs $(u_t^{(i)}, v_t^{(i)})$ by their first coordinate so that $u_t^{(1)} \leq \dots \leq u_t^{(N)}$, preserving the pairing between each $u_t^{(i)}$ and $v_t^{(i)}$. Perform ancestor selection using the Hilbert-ordered weights W_{t-1} and the first coordinate u_t ,

$$a_i = \min \left\{ j : W^{(j)} \geq u_t^{(i)} \right\}, \quad W^{(j)} = \sum_{k=1}^j W_{t-1}^{(\sigma_t(k))}, \quad b_i = \sigma_t(a_i)$$

4 Propagate: Propagate each chosen ancestor b_i through the OU dynamics of Section 3.2, using the predictive mean $\mu_{t|t-1}^{\mathcal{O}_t, (b_i)}$ and covariance $\Sigma_{t|t-1}$ defined in Step 2. The playing-team coordinates are drawn from the predictive marginal by the inverse-CDF transform,

$$x_t^{\mathcal{O}_t, (i)} = \mu_{t|t-1}^{\mathcal{O}_t, (b_i)} + L_{t|t-1} \Phi^{-1} \left(v_t^{(i)} \right)$$

where $L_{t|t-1}$ is the Cholesky factor of the playing-team block $\Sigma_{t|t-1}^{\mathcal{O}_t, \mathcal{O}_t} = \Gamma_{t|t-1}^{\mathcal{O}_t, \mathcal{O}_t} \otimes B$, and Φ^{-1} is the coordinate-wise standard normal quantile, so that the uniform RQMC coordinates $v_t^{(i)}$ are transformed to the Gaussian predictive marginal. Proposition A.2

in Appendix A.2 shows that this transform samples the predictive marginal correctly. The non-playing teams' conditional mean and covariance are updated analytically via the OU prediction and the Kalman conditioning of Section 3.2; the algorithm stores these conditional Gaussian moments rather than explicit coordinates.

5 Compute log-weights: Since the ancestor weights were already used in the inverse-transform resampling, compute the unnormalised log-weights for each particle (i) using only the likelihood function G_t ,

$$\log \tilde{w}_t^{(i)} = \log G_t \left(y_t \mid x_t^{\mathcal{O}_t, (i)} \right)$$

and normalise,

$$w_t^{(i)} = \frac{\tilde{w}_t^{(i)}}{\sum_{n=1}^N \tilde{w}_t^{(n)}}$$

Since the backward step does not resample ancestors, there is no sorting step as in filtering. RB-SQMC smoothing, could in principle, follow the same backward recursion as RB-SMC smoothing (Section 3.4): at each backward step t we instead generate RQMC points $(u_t^{1:M}, v_t^{1:M}) \in \mathbb{R}^{1+2K}$, order them by the conditioning state $X_{t+1}^{*(m)}$, select the Gaussian component by inverse-transform of the backward weights, and draw

$$X_t^{*(m)} = \mu_{t|t+1}^{(I_t)} + L_{t|t+1} \Phi^{-1} \left(v_t^{(m)} \right)$$

where $L_{t|t+1}$ is the Cholesky factor of the backward covariance $\Sigma_{t|t+1}$ and I_t is the selected component index. The caveat is that since the RQMC point set is of dimension $1 + 2K$, the low-discrepancy benefit of RQMC is likely to be negligible relative to Monte Carlo.

3.6 Parameter Estimation

Our static parameters for the model are $\Theta = (\Gamma_0, B, \kappa, \alpha, \beta)$ where $\Gamma_0 \in \mathbb{R}^{K \times K}$, $B \in \mathbb{R}^{2 \times 2}$, $\kappa > 0$, $\alpha \in \mathbb{R}$ and $\beta \in \mathbb{R}$, subject to $\Gamma_0 \succ 0$, $B \succ 0$ and $\det(B) = 1$. We fix the initial mean $\mu_0 = \mathbf{0}_{2K}$, so it is not included in Θ . This is because of an identifiability constraint: if μ_0 were allowed to vary, the model would be invariant to a common shift of all strengths combined with an opposite shift of the intercept α in the likelihood function Equation (3.8). This assumption ties naturally into our model: Given no prior information about teams, we assume that they start from a common baseline of zero strength and the OU process mean-reverts to this baseline

over time.

The smoothing algorithm in Section 3.5 requires the backward-sampled trajectories to be sufficiently diverse to provide a good estimate of the smoothing distribution. The Rao-Blackwellised structure makes the forward filter only partially stochastic: at each step only the four playing-team coordinates are sampled, while the shared covariance $\Sigma_t = \Gamma_t \otimes B$ propagates deterministically and identically across all particles. Smoothing therefore inherits little intrinsic variance, and the diversity of a backward draw is confined to the selected component index I_t and the sampled playing-team means. If the backward weights concentrate on a small number of components, the effective number of distinct smoothed trajectories is small, so the finite-sample estimate of the initial between-team covariance Γ_0 can be unstable or dominated by a few ancestors. Increasing the number of smoothed trajectories M would mitigate this concentration, but it is computationally expensive: each trajectory is sampled sequentially from time T down to 0, and each time step requires evaluating the backward weights against all N particles, giving a cost of $\mathcal{O}(MNT)$ particle-weight evaluations. An attempt was made to run the smoother with a larger number of trajectories, but the resulting cost proved prohibitive within the available budget and the attempt was not successful; this line of investigation was therefore not pursued further.

A simpler and more efficient approach is to estimate the gradient of the marginal log-likelihood. By Fisher’s identity (Del Moral, 2004; Del Moral et al., 2010), the marginal log-likelihood of the particle filter takes the form

$$\nabla_{\Theta} \log p(y_{1:T} \mid \Theta) = \int p(x_{0:T} \mid y_{1:T}, \Theta) \nabla_{\Theta} \log p(x_{0:T}, y_{1:T} \mid \Theta) dx_{0:T},$$

and can be approximated by the score-function estimator of Poyiadjis et al. (2011) defined as

$$\nabla_{\Theta} \log p(y_{1:T} \mid \Theta) \approx \hat{S}_{N,T}(\Theta) = \sum_{i=1}^N w_T^{(i)} \nabla_{\Theta} \log p(\tilde{x}_{0:T}^{(i)}, y_{1:T} \mid \Theta) \quad (3.17)$$

where $w_T^{(i)}$ are the (normalised) particle weights at the final time T and $\tilde{x}_{0:T}^{(i)}$ is the ancestor path of terminal particle i , traced back through the resampling steps. The resampling step would normally make the filter non-differentiable, because the ancestor indices are discrete draws. The stop-gradient construction of Šcibior et al. (2021) rewrites each resampled weight as the ratio $w^{(b_i)} / \text{sg}(w^{(b_i)})$, where b_i is the

selected ancestor and `sg` denotes the stop-gradient operator: on the forward pass this ratio evaluates to one, so the filter is unchanged, while in the gradient the denominator is treated as constant, so resampling contributes no spurious gradient. Backpropagating through $\log \hat{Z}_T$ in this way yields a gradient equal to the consistent score estimator of Poyiadjis et al. (2011), and we maximise $\log p(y_{1:T} \mid \Theta)$ by gradient ascent on the static parameters Θ .

3.7 Performance comparison

We compare RB-SQMC against a factorial state-space baseline, specifically the biased Gaussian extended Kalman filter (EKF) of `cuthberto-carlos` (Duffield and Tan, 2026), on the 2026 World Cup. This is an end-to-end comparison of two complete pipelines: the EKF is a biased Gaussian approximation, whereas RB-SQMC is a particle approximation, so any difference reflects the effectiveness of the different inference approach. We evaluate the model performance on three metrics: marginal log-likelihood from training, prediction outputs and comparison with FIFA rankings.

3.7.1 Experimental design

The historical data is resctricted to the $K = 48$ teams that played in the 2026 World Cup. It is split chronologically into a training period [1980-01-01, 2024-01-01), a test period [2024-01-01, 2026-06-11), and a prediction period from 11 June 2026. The prediction period contains the 104 World Cup 2026 fixtures used for the headline scoring. Matches with scores exceeding the truncation threshold of `max_goals = 8` are excluded before the split.

3.7.2 Training: marginal log-likelihood

The static parameters $\Theta = (\Gamma_0, B, \kappa, \alpha, \beta)$ are estimated by maximising the marginal log-likelihood as mentioned in Section 3.6. The SQMC log-likelihood is a particle estimate, its gradient is averaged over independently randomised RQMC replicas at each update, whereas the EKF gradient is deterministic. RB-SQMC evaluates the same likelihood on the sampled playing-team coordinates; the EKF integrates the bivariate-Poisson likelihood under the predictive Gaussian by Gauss–Hermite quadrature of degree 32 (Duffield, 2024). The full parameter set and the exact

configuration are listed in Appendix C. Table 3.1 reports the training and test log-likelihoods.

	EKF	RB-SQMC
Final train log-likelihood	-15675.04	-14649.74
Final test log-likelihood	-1040.01	-1005.42
Maximum test log-likelihood (epoch)	-1040.01 (50)	-959.96 (49)

Table 3.1: Training and test log-likelihoods for the EKF and RB-SQMC at the pre-specified final epoch. Higher (less negative) values indicate a better fit.

RB-SQMC attained a higher final train log-likelihood than the EKF (-14649.74 versus -15675.04) and a higher final test log-likelihood (-1005.42 versus -1040.01). A point to note is that the log-likelihood from the EKF is a Gaussian-approximate normalising constant from moment filtering while the SQMC value is a particle estimate with finite-sample error, so the raw gap is not an exact measure of model fit, and the raw results here do not necessarily indicate superiority. Figure 3.2 shows the training and test log-likelihood trajectories across epochs for both methods.

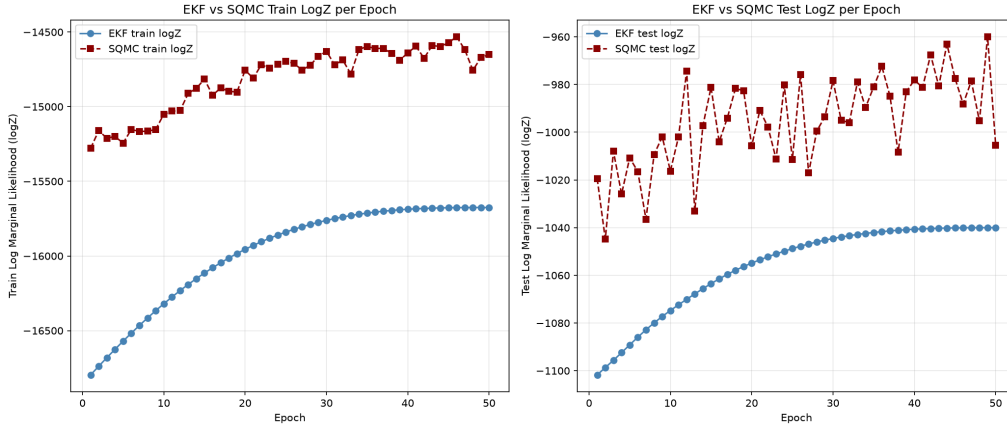


Figure 3.2: Training and test log-likelihood against epoch for the EKF and RB-SQMC. The EKF value is a Gaussian-approximate normalising constant; the SQMC value is a particle estimate.

The SQMC test log-likelihood is also markedly less stable across epochs than the EKF's. The EKF trajectory is smooth and monotone, improving steadily from -1101.76 to -1040.01 over the 50 updates. The SQMC trajectory ranges between approximately -1044.84 and -959.96 . This instability comes from SQMC's Monte Carlo error; the maximum test value is a favourable draw rather than a robust optimum.

3.7.3 Prediction outputs

Both methods produce a one-step-ahead score grid for the 104 matches. We score the resulting distributions with the mean Brier score over the three outcomes (home win, draw, away win), the mean predictive log-score, and outcome accuracy of the 104 World Cup 2026 fixtures. The Brier score is defined as,

$$\text{Brier score} = \frac{1}{N} \sum_{i=1}^N \sum_{c \in \{H,D,A\}} (p_{ic} - o_{ic})^2$$

where p_{ic} is the predicted probability of outcome c for match i , and o_{ic} is the observed outcome indicator. The inner sum is over the three outcomes (home win, draw, away win), so the uniform reference is $2/3$ under this convention. The predictive log-score is defined as,

$$\text{Predictive log-score} = \frac{1}{N} \sum_{i=1}^N \log p(y_i^{\text{home}}, y_i^{\text{away}})$$

which is the average log-probability assigned to the observed outcome o_i for match i . Outcome accuracy is defined as the proportion of matches for which the predicted outcome with the highest probability matches the observed outcome. Table 3.2 reports the results, and the full win-loss predictions for the fixtures and the knockout stage are collected in Appendix D.

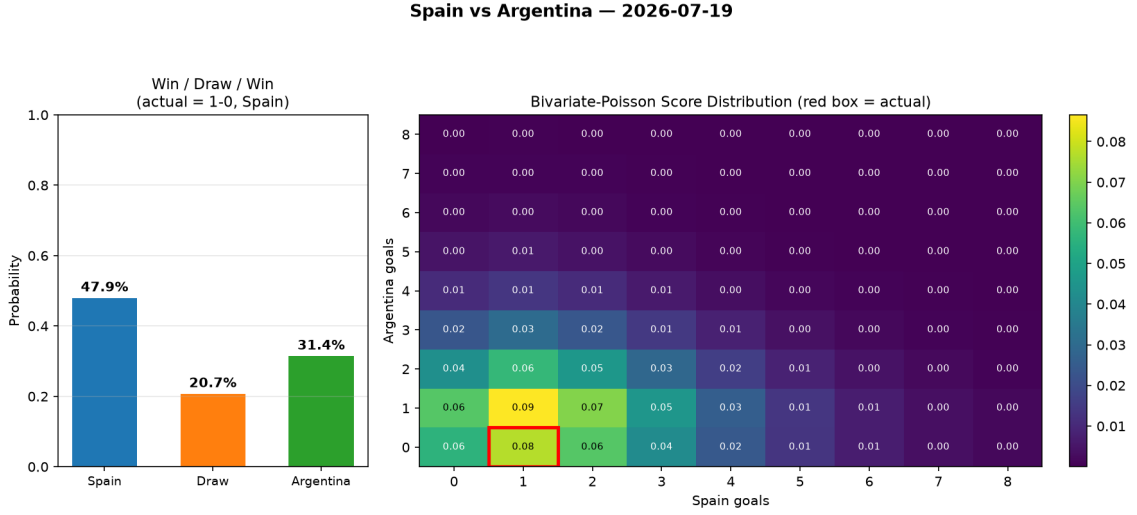
	EKF	RB-SQMC
Mean Brier score	0.5442	0.6878
Uniform reference Brier score	2/3	2/3
Brier skill score vs. uniform	0.1838	−0.0316
Mean predictive log-score	−3.131	−3.664
Outcome accuracy	0.5288	0.4712
Exact-score accuracy	0.0769	0.0481

Table 3.2: One-step-ahead prediction metrics on the 104 World Cup 2026 fixtures. The Brier skill score is relative to the uniform reference $2/3$ and lower Brier scores indicate better prediction on Home Win/Draw/Away Win. The predictive log-score is the mean scoreline log-probability over the truncated score grid.

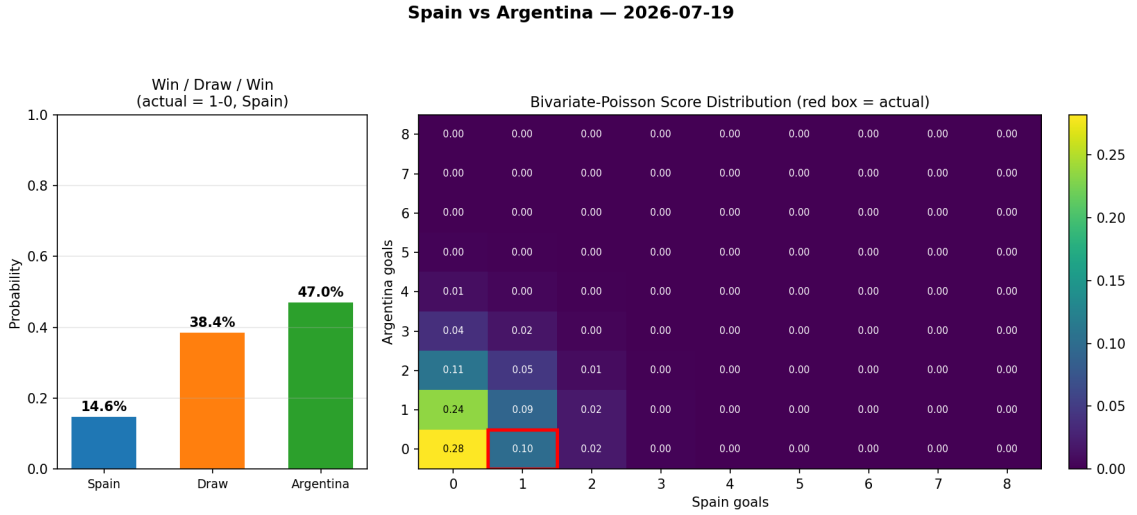
On the 104 prediction-split matches, RB-SQMC produces worse performance predictive performance than the EKF on all metrics. RB-SQMC also performs worse

than the uniform reference of Brier score, which means that it is worse than guessing equally on all three outcomes of a match.

As an illustration of a forecast, we review the final of the 2026 World Cup, Spain vs. Argentina, played on 19 July 2026. The images in Figure 3.3 show the pre-match predicted score grids for both methods, with the actual result (a 1–0 win for Spain) marked by a red outline. The EKF assigns the highest probability to a home win (0.479) and a draw (0.207), with the modal score at 1–1. RB-SQMC instead assigns the highest probability to an Argentina win (0.470), then a draw (0.384) and a Spain win (0.146), with the modal score at 0–0. The EKF model had a better Brier score of 0.413 compared to RB-SQMC’s 1.097, which favoured an Argentina win heavily. The predicted results for the knockout stages can be found in Appendix D, and all 104 predictions are recorded in the repository at a commit-specific path listed in Appendix B.



(a) EKF predicted score grid.



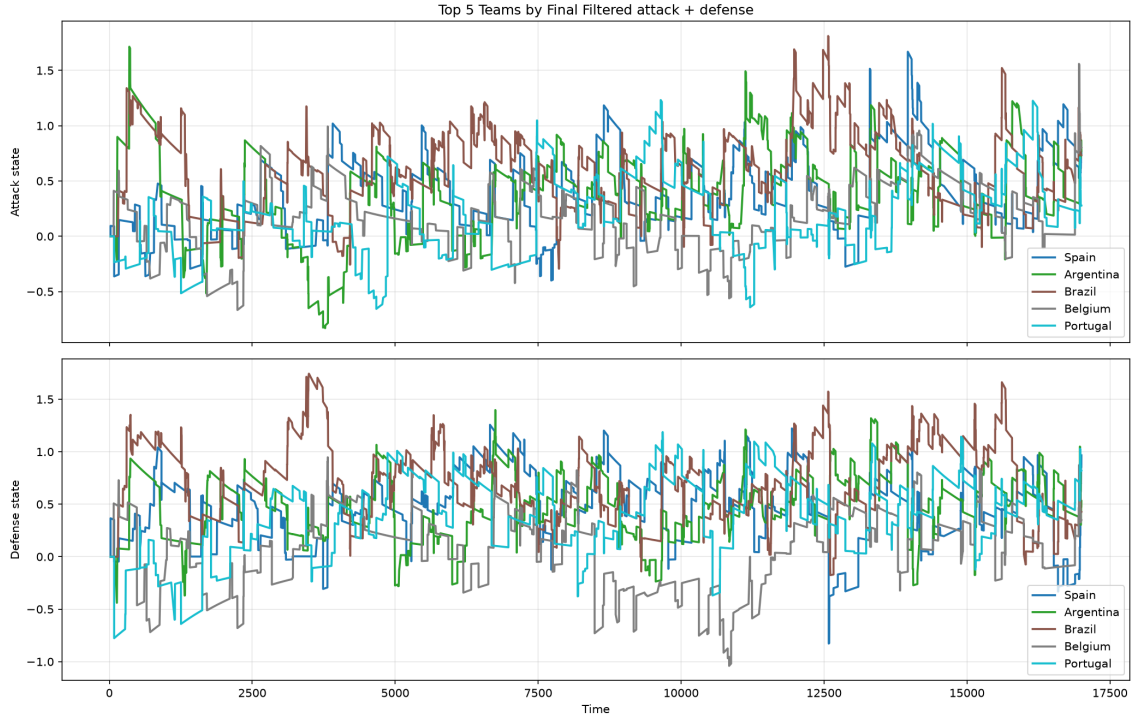
(b) RB-SQMC predicted score grid.

Figure 3.3: Predicted score grids for the final match, Spain vs. Argentina, under the EKF and RB-SQMC. The actual result is marked by a red outline.

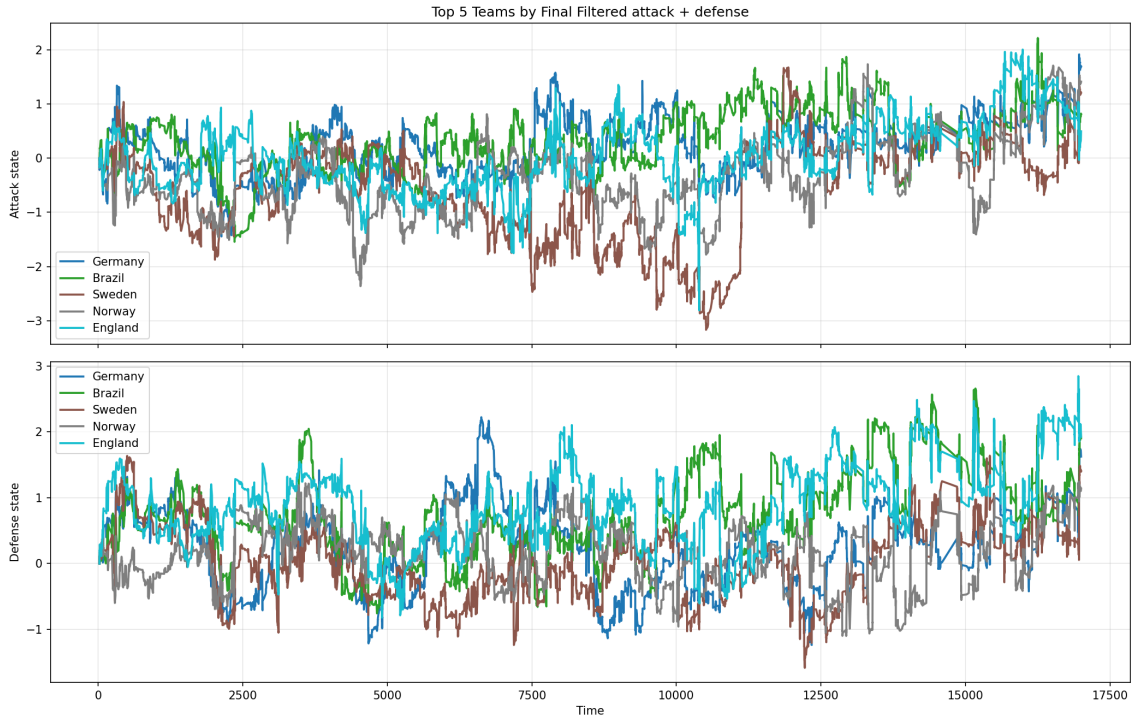
3.7.4 Evaluation

The EKF produces a single deterministic trajectory, thus the filtering states are linear, whereas RB-SQMC reports the weighted particle mean of the posterior, which appear to be more stochastic, representing the random evolution of teams. Figure 3.4 shows the filtered attacking and defensive strength trajectories for the final top-five teams under each method.

The difference between EKF and RB-SQMC is that RB-SQMC retains the cross-team covariance, whereas the EKF discards it after every match. Figure 3.5 reports the



(a) EKF filtered states.



(b) RB-SQMC filtered states.

Figure 3.4: Filtered attacking and defensive strengths over time for the final top-five teams under the EKF and RB-SQMC.

strongest positive and negative between-team correlations carried by the prior filter covariance Γ_0 . The strongest positive pairs are dominated by teams in the same confederation, such as England–France and Argentina–Colombia, which is consistent with the assumption that regional teams evolve together.

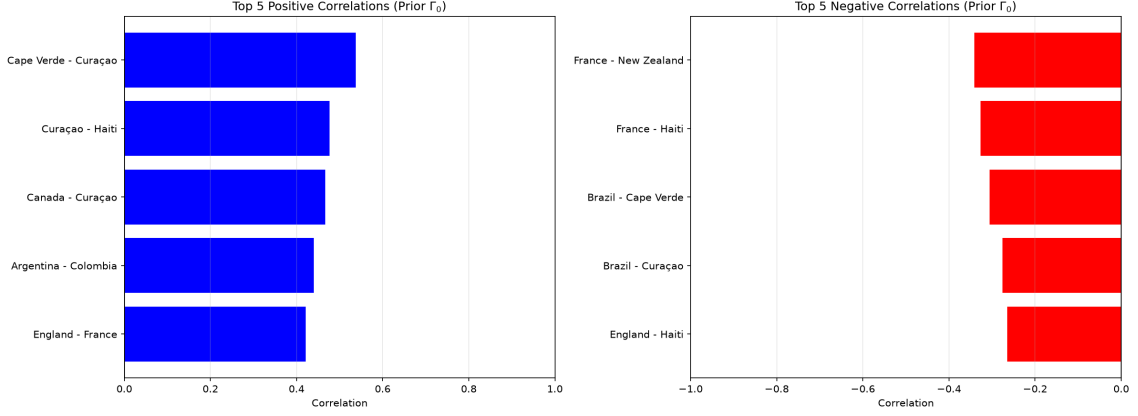


Figure 3.5: Strongest positive and negative between-team correlations carried by the prior filter covariance Γ_0 under RB-SQMC.

3.7.5 Comparison with the FIFA ranking

We compare the post-tournament rankings of both methods with the FIFA Men’s World Ranking (FIFA, 2026), which uses the Elo-stye rating system of Elo (1978) to rank national teams, used commonly in chess and go. Table 3.3 reports the top ten teams under each method after the World Cup, together with the FIFA ranking published on 20 July 2026.

Rank	EKF	RB-SQMC	FIFA
1	Spain	Germany	Spain
2	Argentina	Brazil	Argentina
3	Brazil	Sweden	France
4	Belgium	Norway	England
5	Portugal	England	Brazil
6	Colombia	Argentina	Morocco
7	England	Australia	Portugal
8	Morocco	Ivory Coast	Belgium
9	Switzerland	Canada	Netherlands
10	France	Netherlands	Mexico

Table 3.3: Post-tournament top-ten team rankings under the EKF and RB-SQMC, compared with the FIFA Men’s World Ranking of 20 July 2026 (FIFA, 2026).

The EKF’s post-tournament ranking aligns closely with the FIFA ranking: both place Spain first and Argentina second, and both include France, England, Brazil, Morocco, Portugal and Belgium in the top ten. RB-SQMC’s ranking is less plausible: it omits Spain, France, Portugal and Belgium from the top ten entirely despite these teams advancing to the top 8 of the 2026 World Cup. It also places Argentina sixth despite Argentina reaching the final and Germany first, despite Germany being eliminated in the round of 32. The EKF therefore produces the more justifiable qualitative ranking, along with its better prediction metrics, even though RB-SQMC attains the higher log-likelihood. The full pre- and post-tournament rankings for both methods are collected in Appendix E.

3.7.6 Discussion

For our dataset of 1980 onwards, RB-SQMC attained a higher test log-likelihood than the EKF, yet it produced worse predictive performance on the 104 World Cup 2026 fixtures. We highlight some key points for discussion on the issues we faced.

Computational cost is the principal limitation The factorial SSM is extremely efficient: it runs on a CPU in seconds and can be run on a laptop. In contrast, a single optimisation run of RB-SQMC with 512 particles took approximately 1.6 hours on an A100 GPU, dominated by the repeated particle-filter evaluations and the Hilbert sorting. This cost prevented running multiple seeds or multiple RQMC

scrambles, so the reported SQMC results are a single finite-particle realisation. A fixed-checkpoint sensitivity analysis over particle counts and random seeds would be needed to establish numerical stability, and averaging over several RQMC scrambles would give a more reliable and robust estimate of RB-SQMC’s predictive performance.

An approximate model is suitable for this problem given the constraints

Particle methods are more flexible for non-Gaussian likelihoods than the Gaussian moment approximation of the EKF, and the Rao-Blackwellised construction retains the cross-team covariance that the factorial EKF discards. Empirically,, this additional flexibility did not translate into better prediction on the World Cup fixtures. We also observed that filter states of the EKF was similar to the Fifa 2026 rankings. One explanation is that the independent-evolution and Gaussian assumptions of the factorial model are adequate for this data set. The RB-SQMC construction may be better suited to settings in which the latent states are more strongly non-Gaussian, for example in finance or biology, but this is a hypothesis to be tested rather than a conclusion of the present study.

Chapter 4

Conclusion

We developed the framework for performing SQMC on parallel accelerators in `jax` and we observe that SQMC does have a performance advantage when implemented on parallel accelerators as compared to CPU. We also demonstrated the application of SQMC to a football model, which is a high-dimensional problem with sparse local observations. However, the final RB-SQMC model does not perform better than a factorial state-space extended Kalman filter (EKF) model in this comparison.

These implementations motivate two directions for future work.

- **RB-SQMC smoothing with low discrepancy preserved.** In Section 3.5, we briefly described Rao–Blackwellised SQMC smoother. The RQMC points lie in high dimension, however, so their low-discrepancy advantage over Monte Carlo is negligible. Future work should develop smoothing for SQMC with asynchronous, sparse local observations that retains the low-discrepancy benefit of RQMC.
- **Computationally cheaper parameter estimation for SQMC.** Particle-based parameter estimation for SQMC is limited by computational cost, as Section 3.7 shows. Potential alternative algorithms include fixed-lag approximations such as particle stitching (Duffield and Singh, 2022), expectation–maximisation with closed-form smoothing (Duffield et al., 2024), and memory-reducing methods such as the marginal particle filter (Klaas et al., 2005) or PaRIS (Olsson and Westerborn, 2017). However, these algorithms assume random particle systems and it remains to establish whether—and with what cost and bias—they remain effective when resampling is replaced by the quantisation-based, low-discrepancy resampling on which SQMC relies, and whether they extend to asynchronous and sparse observations.

This work forms part of the wider body of work within the open-source state-space inference package `cuthbert`.

Appendix A

Proofs

This appendix contains the proofs deferred from Chapter 3.

A.1 Kronecker structure reduces the inference dimension

This appendix proves the claim underlying the Kronecker parameterisation of the initial covariance in Section 3.2. The result is referenced in the model section of Chapter 3.

Proposition A.1. *Let $\Sigma_0 = \Gamma_0 \otimes B$ with $\Gamma_0 \in \mathbb{R}^{K \times K}$, $B \in \mathbb{R}^{2 \times 2}$ symmetric and positive definite, and $\det(B) = 1$. Then the Kronecker parameterisation reduces the number of free parameters of the initial covariance from $K(2K + 1)$ for a general symmetric $2K \times 2K$ matrix to $K(K + 1)/2 + 2$. The reduction is a factor of approximately four, but remains of order $\mathcal{O}(K^2)$.*

Proof. A general symmetric $2K \times 2K$ covariance matrix has $2K(2K + 1)/2 = K(2K + 1)$ free entries. Under the Kronecker form $\Sigma_0 = \Gamma_0 \otimes B$, the parameterisation is determined by the symmetric matrix $\Gamma_0 \in \mathbb{R}^{K \times K}$, which contributes $K(K + 1)/2$ free entries, together with the 2×2 symmetric matrix B , which contributes 3 entries subject to the single constraint $\det(B) = 1$, i.e. 2 free parameters. The total is therefore $K(K + 1)/2 + 2$. The main benefit is computational rather than parametric: in Chapter 3, the Kalman recursions are carried out on the $K \times K$ matrix Γ rather than the full $2K \times 2K$ matrix Σ . Distinguishing the costs: the shared covariance requires $\mathcal{O}(K^2)$ storage; the per-observation update performs a

2×2 Schur solve, dense rank-two products of order $\mathcal{O}(K^2)$ for the covariance update, and N particle-mean updates of order $\mathcal{O}(NK)$; no $2K \times 2K$ solve is ever formed. $\square$

A.2 The deterministic inverse-CDF transform in RB-SQMC

This appendix proves the claim underlying the propagation step of RB-SQMC (Section 3.5). Under the Ornstein–Uhlenbeck dynamics of Section 3.2, each Rao–Blackwellised component is first advanced by the deterministic mean-reverting prediction, and the playing-team block is then drawn from its predictive marginal by the inverse-CDF transform

$$x_t^{\mathcal{O}_t, (i)} = \mu_{t|t-1}^{\mathcal{O}_t, (b_i)} + L_{t|t-1} \Phi^{-1}(v_t^{(i)}), \quad (\text{A.1})$$

where $\mu_{t|t-1}^{\mathcal{O}_t, (b_i)} = \mu_0^{\mathcal{O}_t} + \phi_t(\mu_{t-1}^{\mathcal{O}_t, (b_i)} - \mu_0^{\mathcal{O}_t})$, $L_{t|t-1}$ is the Cholesky factor of the predictive covariance $\Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t}$, and $v_t^{(i)} \in [0, 1]^4$ is a quasi-random point.

Proposition A.2. *Let $v \sim \text{Unif}([0, 1]^4)$ with independent coordinates, let Φ denote the standard normal CDF, and let $\Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t}$ be symmetric positive definite with Cholesky factor $L_{t|t-1}$, i.e. $L_{t|t-1} L_{t|t-1}^\top = \Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t}$. Let $\mu_{t|t-1}^{\mathcal{O}_t, (b)}$ be the predictive mean of the playing-team block for ancestor mean $\mu_{t-1}^{\mathcal{O}_t, (b)}$, given by*

$$\mu_{t|t-1}^{\mathcal{O}_t, (b)} = \mu_0^{\mathcal{O}_t} + \phi_t(\mu_{t-1}^{\mathcal{O}_t, (b)} - \mu_0^{\mathcal{O}_t}), \quad (\text{A.2})$$

with the predictive covariance

$$\Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t} = \phi_t^2 \Sigma_{t-1}^{\mathcal{O}_t \mathcal{O}_t} + (1 - \phi_t^2) \Sigma_0^{\mathcal{O}_t \mathcal{O}_t}. \quad (\text{A.3})$$

Then

$$x = \mu_{t|t-1}^{\mathcal{O}_t, (b)} + L_{t|t-1} \Phi^{-1}(v) \quad (\text{A.4})$$

is distributed as $\mathcal{N}(\mu_{t|t-1}^{\mathcal{O}_t, (b)}, \Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t})$, the predictive marginal of the playing-team block under the bootstrap proposal.

Proof. The argument proceeds in two steps: the probability integral transform, followed by an affine transformation of a standard Gaussian vector.

Step 1: coordinatewise quantile transform. By definition of the distribution function, the random variable $\Phi(Z_j)$ with $Z_j \sim \mathcal{N}(0, 1)$ is uniformly distributed on

$[0, 1]$, since

$$\mathbb{P}(\Phi(Z_j) \leq u) = \mathbb{P}(Z_j \leq \Phi^{-1}(u)) = \Phi(\Phi^{-1}(u)) = u, \quad u \in [0, 1].$$

Because Φ is strictly increasing and continuous, the same argument in reverse gives the probability integral transform: if $V_j \sim \text{Unif}([0, 1])$, then

$$Z_j := \Phi^{-1}(V_j) \sim \mathcal{N}(0, 1).$$

Applying this coordinatewise to the independent coordinates of v , the vector

$$Z := \Phi^{-1}(v) = (\Phi^{-1}(v_1), \dots, \Phi^{-1}(v_4))$$

has independent standard normal components, so $Z \sim \mathcal{N}(0, I_4)$.

Step 2: affine transformation. For any matrix A and any vector c , the affine map $Z \mapsto c + AZ$ sends a standard Gaussian vector to $\mathcal{N}(c, AA^\top)$. Taking $c = \mu_{t|t-1}^{\mathcal{O}_t, (b)}$ and $A = L_{t|t-1}$, and using $L_{t|t-1}L_{t|t-1}^\top = \Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t}$, we obtain

$$x = \mu_{t|t-1}^{\mathcal{O}_t, (b)} + L_{t|t-1}Z \sim \mathcal{N}(\mu_{t|t-1}^{\mathcal{O}_t, (b)}, L_{t|t-1}L_{t|t-1}^\top) = \mathcal{N}(\mu_{t|t-1}^{\mathcal{O}_t, (b)}, \Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t}).$$

It remains to justify the two moments. The mean $\mu_{t|t-1}^{\mathcal{O}_t, (b)}$ is the deterministic OU prediction of the playing-team block, as stated. The covariance is the sum of the OU transition noise and the propagated uncertainty of the carried component: the OU update in Section 3.2 contributes $(1 - \phi_t^2)\Sigma_0$ (whose playing-team block is $(1 - \phi_t^2)\Sigma_0^{\mathcal{O}_t \mathcal{O}_t}$), while the prior component covariance Σ_{t-1} is carried forward by the mean-reverting factor, contributing $\phi_t^2\Sigma_{t-1}$, with playing-team block $\phi_t^2\Sigma_{t-1}^{\mathcal{O}_t \mathcal{O}_t}$. Their sum is exactly the stated $\Sigma_{t|t-1}^{\mathcal{O}_t \mathcal{O}_t}$. Hence the transform samples the predictive marginal under the bootstrap proposal. $\square$

A remark on the Monte Carlo analogue is in order. When the $v_t^{(i)}$ are IID uniform draws, this transform is merely the usual reparameterisation of the bootstrap proposal, and the resulting $x_t^{\mathcal{O}_t, (i)}$ are an exact Monte Carlo sample from the playing-team marginal $q_t^{(b)}$ defined in Section 3.3. In RB-SQMC the same map is applied to a quasi-random point set $v_t^{(1:N)}$ instead. Since the map is deterministic and invertible, the low-discrepancy structure of the point set carries over to the playing-team coordinates; this is the sense in which the effective propagation dimension is the dimension of v , namely four. The transform is exact in distribution; the only approximation in RB-SQMC is the finite quasi-Monte Carlo error in the resampling step, not this

propagation map.

Appendix B

Code

This appendix records how the results in this dissertation can be reproduced. The implementation lives in the public repository `ryantjx/sqmc-cuthbert`; it is part of the process of being integrated into `state-space-models/cuthbert` under issue #255. The repository is organised as two packages: `sqmc/`, containing the shared SQMC library (QMC point generation, the SQMC filter and Hilbert sorting) together with its CPU/GPU benchmark suite, and `rbsqmc/`, containing the football application (data preparation, the RB-SMC, RB-SQMC and factorial EKF filters, and the comparison pipelines). The football package imports the QMC generation and Hilbert-sorting modules from the SQMC package at runtime. The numerical results reported in this dissertation are committed at the repository root under `results/`: `results/qmc_hilbert_sqmc/` holds the Chapter 2 accelerator benchmarks and `results/sqmc_ekf/` the Chapter 3 football comparison. These are the latest results from the authoritative runs described in Appendix C.

B.1 Replication

The experiments require Python 3.13 or later with the dependencies listed in the repository’s `requirements.txt`. GPU runs use the CUDA 12 build of `jax`; the football code selects its platform through the `RBSQMC_PLATFORM` environment variable. Two data files are generated on a fresh clone: the Sobol’ direction-number table, built from the committed Joe-Kuo source table by `_generate_sobol_data.py`, and the parquet cache of the international football results, downloaded from the `martj42/international-results` source. All commands are run from the repository root.

Table B.1 maps each empirical chapter to the entry point that reproduces it. The performance comparison of Chapter 2 is produced by three standalone benchmarks, which can be run directly on a CPU or GPU host or through the Colab launcher, which provisions an A100 session, executes the three stages sequentially and verifies each stage’s artefacts before proceeding. The football comparison of Chapter 3 trains the correlated RB-SQMC model and the factorial EKF baseline on identical data splits and optimiser settings under the configuration recorded in Appendix C, and evaluates one-step-ahead predictions on the held-out period. Each run writes a timestamped output directory containing the effective configuration, device and software metadata, the git commit, source hashes, results and figures, so that any reported number can be traced to its generating configuration.

Result	Entry point	Output
QMC generation (Sec. 2.2.2)	<code>sqmc.comparison.benchmark_qmc</code>	<code>ryantjx/sqmc-cuthbert/sqmc/comparison/outputs/</code>
Hilbert sorting (Sec. 2.2.3)	<code>sqmc.comparison.benchmark_hilbert_sort</code>	<code>ryantjx/sqmc-cuthbert/sqmc/comparison/outputs/</code>
SQMC CPU/GPU (Sec. 2.2.4)	<code>sqmc.comparison.benchmark_sqmc</code>	<code>ryantjx/sqmc-cuthbert/sqmc/comparison/outputs/</code>
RB-SQMC vs EKF (Sec. 3.7)	<code>rbsqmc.comparison.sqmc_ekf.run</code>	<code>ryantjx/sqmc-cuthbert/rbsqmc/comparison/sqmc_ekf/outputs/</code>

Table B.1: Entry points reproducing the empirical results. Each writes a timestamped directory with configuration, provenance metadata and results.

The authoritative football run is 09092026_1431 at on an NVIDIA A100, as recorded in Appendix C. The benchmark suite records the same provenance metadata (device descriptions, software versions, git commit and hashes of the benchmark sources and the Sobol’ direction data) in every run directory. Timed measurements use ten repetitions after two warm-up calls, with compilation and input transfers excluded

from the reported runtimes; the SQMC accuracy comparison selects particle counts under a per-trajectory runtime budget using selection randomisations and reports accuracy from separate validation randomisations, as described in Section 2.2.1.

Appendix C

Configuration

This appendix records the exact configuration used for the performance comparison in Section 3.7. Both methods consume the same frozen, chronologically ordered data set and the same optimiser. The comparison is end to end: the two pipelines differ in prior dependence, posterior factorisation, likelihood integration and stochastic versus deterministic objective as well as in the correlated Rao-Blackwellised construction, so any difference between them is interpreted as a difference between complete pipelines and is not attributed to a single component.

C.1 Data and split

The data set is the frozen international football results file `results.csv` from `martj42/international-results` (licence CC-BY 4.0, retrieved 9 September 2026), containing 49,521 rows with one row per match; the frozen file’s SHA-256 checksum is recorded in the repository’s data-preparation module at commit `336842f`. The rows are split chronologically into a training period [1980-01-01, 2024-01-01), a test period [2024-01-01, 2026-06-11), and a prediction period from 11 June 2026. The training set contains 4,702 matches, the test set 305, and the prediction set 104. Matches with scores exceeding the truncation threshold `max_goals = 8` are excluded before the split, and same-day repeats are retained as separate rows with zero elapsed time. The historical rows are restricted to matches involving the 48 tournament teams, a deliberate selection decision documented in Section 3.7.

C.2 Model parameters

The static parameters are $\Theta = (\Gamma_0, B, \kappa, \alpha, \beta)$, with $\Gamma_0 \in \mathbb{R}^{K \times K}$, $B \in \mathbb{R}^{2 \times 2}$, $\kappa > 0$, $\alpha \in \mathbb{R}$ and $\beta \in \mathbb{R}$, subject to $\Gamma_0 \succ 0$, $B \succ 0$ and $\det(B) = 1$. The number of teams is $K = 48$. The initial mean is $\mu_0 = \mathbf{0}_{2K}$.

C.3 Optimisation

Both methods use Adam with a cosine-decay learning-rate schedule, learning rate 0.02, seed 0 and exactly 50 updates, with the final-epoch checkpoint pre-specified. The SQMC log-likelihood gradient is averaged over 15 independently randomised RQMC replicas at each update; the EKF gradient is deterministic. The EKF integrates the bivariate-Poisson likelihood under the predictive Gaussian by Gauss–Hermite quadrature of degree 32. The unconstrained-to-constrained parameterisation maps Γ_0 and B through Cholesky factors with a fixed jitter floor on the diagonal, κ through a softplus transform, and α, β as unconstrained; initial values are zeros for the unconstrained parameters. The epoch count was fixed in advance rather than chosen by a convergence criterion.

C.4 Software environment

The reported runs used Python 3.13 with `jax` and `jaxlib` built for CUDA 12 on the A100 host, and the CPU build on the Intel Xeon host; the exact package versions, CUDA and XLA versions, operating system, floating-point precision (`float64` for the football comparison), compilation policy (`jax.jit` with warm-up calls excluded from timing), device-synchronisation call, host–device placement and thread settings are recorded in the provenance metadata written by every run directory, as described in Appendix B. CPU and GPU runs were executed in the same process for the shared-implementation benchmarks of Chapter 2.

C.5 Full configuration

Table C.1 lists the complete configuration. The values correspond to the authoritative comparison run 09092026_1431 at source commit 336842f, which is the run reported in Section 3.7.

Setting	Value	Description
Number of teams K	48	Teams in the World Cup 2026 model
Particles N	512	SQMC particle count
Epochs	50	Optimisation updates
Learning rate	0.02	Adam step size
Learning-rate schedule	cosine decay	Decay of the learning rate over epochs
Seed	0	Random seed for optimisation and RQMC
SQMC gradient replicas	15	RQMC replicas averaged per gradient
Gauss-Hermite degree	32	Quadrature degree for the EKF likelihood
<code>max_goals</code>	8	Score truncation threshold
<code>match_scale</code>	1	Observation scale for non-friendly matches
Include friendlies	yes	Friendly fixtures retained in training
Training start	1980-01-01	Start of the training period
Test start	2024-01-01	Start of the test period
Prediction start	2026-06-11	Start of the prediction period
GPU	A100	Hardware for the SQMC run
Source commit	336842f	Repository revision of the run
Run ID	09092026_1431	Identifier of the authoritative run

Table C.1: Configuration for the performance comparison of Section 3.7.

Appendix D

Prediction

The tables below report the predicted scoreline for each of the 31 knockout-stage fixtures of World Cup 2026 under both methods described in Section 3.7, together with the actual result. The group-stage fixtures are omitted here for space and are recorded in the repository at the commit-specific path listed in Appendix B. For each match, the table reports the modal scoreline under each method, that is, the scoreline (h, a) with the highest probability in the one-step-ahead score grid; this can disagree with the modal three-way outcome (home win, draw, away win), which is the quantity scored by the Brier score and outcome accuracy in Section 3.7.

D.1 Round of 32

Match	Actual	EKF	RB-SQMC
South Africa–Canada	0–1	0–1	0–3
Brazil–Japan	2–1	1–0	1–0
Germany–Paraguay	1–1	1–0	5–0
Netherlands–Morocco	1–1	1–1	1–0
Ivory Coast–Norway	1–2	1–1	1–1
France–Sweden	3–0	3–1	0–5
Mexico–Ecuador	2–0	0–0	0–0
England–DR Congo	2–1	0–0	1–0
Belgium–Senegal	3–2	2–0	0–1
United States–Bosnia and Herzegovina	2–0	2–1	3–1
Spain–Austria	3–0	1–1	1–1
Portugal–Croatia	2–1	2–0	2–2
Switzerland–Algeria	2–0	2–1	1–0
Australia–Egypt	1–1	0–0	0–0
Argentina–Cape Verde	3–2	1–0	2–0
Colombia–Ghana	1–0	2–0	1–1

Table D.1: Outcome and modal-score predictions of the bivariate-Poisson model for the Round of 32. The modal scoreline is the scoreline with the highest probability in the one-step-ahead score grid.

D.2 Round of 16

Match	Actual	EKF	RB-SQMC
Canada–Morocco	0–3	0–0	1–0
Paraguay–France	0–1	0–2	0–2
Brazil–Norway	1–2	3–1	0–0
Mexico–England	2–3	1–0	0–0
Portugal–Spain	0–1	1–0	0–0
United States–Belgium	1–4	1–2	2–3
Argentina–Egypt	3–2	2–0	0–0
Switzerland–Colombia	0–0	1–0	3–0

Table D.2: Outcome and modal-score predictions of the bivariate-Poisson model for the Round of 16. The modal scoreline is the scoreline with the highest probability in the one-step-ahead score grid.

D.3 Quarter-finals

Match	Actual	EKF	RB-SQMC
France–Morocco	2–0	1–0	0–0
Spain–Belgium	2–1	1–1	1–1
Norway–England	1–2	1–1	0–0
Argentina–Switzerland	3–1	1–1	0–0

Table D.3: Outcome and modal-score predictions of the bivariate-Poisson model for the Quarter-finals. The modal scoreline is the scoreline with the highest probability in the one-step-ahead score grid.

D.4 Semi-finals

Match	Actual result	EKF predicted	RB-SQMC predicted
France–Spain	0–2	1–0	1–1
England–Argentina	1–2	1–1	0–0

Table D.4: Outcome and modal-score predictions of the bivariate-Poisson model for the Semi-finals. The modal scoreline is the scoreline with the highest probability in the one-step-ahead score grid.

D.5 3rd place

Match	Actual result	EKF predicted	RB-SQMC predicted
France–England	4–6	1–0	0–1

Table D.5: Outcome and modal-score predictions of the bivariate-Poisson model for the 3rd-place match. The modal scoreline is the scoreline with the highest probability in the one-step-ahead score grid.

D.6 Final

Match	Actual result	EKF predicted	RB-SQMC predicted
Spain–Argentina	1–0	1–1	0–0

Table D.6: Outcome and modal-score predictions of the bivariate-Poisson model for the Final. The modal scoreline is the scoreline with the highest probability in the one-step-ahead score grid.

Appendix E

Rankings

This appendix collects the top-twenty pre- and post-tournament team rankings produced by the EKF and RB-SQMC methods described in Section 3.7, together with the FIFA Men’s World Ranking (FIFA, 2026) used as an external descriptive reference in Section 3.7. The model rankings are obtained by ordering the teams by their filtered total strength, the sum of the attacking and defensive components, at the relevant time point; a larger defence coordinate represents stronger defence, so the sum ranks attacking and defensive quality jointly. The top twenty are shown for compactness rather than the full rankings of all 48 teams; the complete rankings are recorded in the repository at the commit-specific path listed in Appendix B.

E.1 Pre-tournament rankings

Table E.1 reports the top twenty teams before the World Cup under each method. No external pre-tournament ranking is used in this dissertation.

Rank	EKF	RB-SQMC
1	Belgium	Portugal
2	Brazil	Belgium
3	Switzerland	England
4	Colombia	France
5	Austria	Croatia
6	Portugal	Japan
7	France	Czech Republic
8	Algeria	Netherlands
9	Ecuador	Austria
10	Japan	Morocco
11	Argentina	Norway
12	Australia	Brazil
13	Germany	Argentina
14	Mexico	Spain
15	Canada	Colombia
16	Netherlands	Algeria
17	Morocco	Canada
18	Ivory Coast	Senegal
19	Spain	Scotland
20	Norway	Ecuador

Table E.1: Pre-tournament top-twenty team rankings under the EKF and RB-SQMC.

E.2 Post-tournament rankings

Table E.2 reports the top twenty teams after the World Cup under each method, together with the FIFA ranking of 20 July 2026.

Rank	EKF	RB-SQMC	FIFA
1	Spain	Germany	Spain
2	Argentina	Brazil	Argentina
3	Brazil	Sweden	France
4	Belgium	Norway	England
5	Portugal	England	Brazil
6	Colombia	Argentina	Morocco
7	England	Australia	Portugal
8	Morocco	Ivory Coast	Belgium
9	Switzerland	Canada	Netherlands
10	France	Netherlands	Mexico
11	Mexico	Senegal	Colombia
12	Netherlands	Switzerland	Germany
13	Norway	Belgium	Croatia
14	Japan	Japan	Switzerland
15	Senegal	Egypt	Italy
16	Canada	Spain	United States
17	Australia	Croatia	Japan
18	Egypt	Portugal	Senegal
19	United States	Ecuador	Norway
20	Germany	United States	Uruguay

Table E.2: Post-tournament top-twenty team rankings under the EKF and RB-SQMC, compared with the FIFA Men’s World Ranking of 20 July 2026 (FIFA, 2026).

Bibliography

- Peter Bickel, Bo Li, and Thomas Bengtsson. Sharp failure rates for the bootstrap particle filter in high dimensions. In Bert Clarke and Subhashis Ghosal, editors, *Pushing the Limits of Contemporary Statistics: Contributions in Honor of Jayanta K. Ghosh*, volume 3 of *Institute of Mathematical Statistics Collections*, pages 318–329. Institute of Mathematical Statistics, 2008. doi: 10.1214/074921708000000228.
- Arthur R. Butz. Convergence with Hilbert’s space filling curve. *Journal of Computer and System Sciences*, 3(2):128–146, 1969.
- Nicolas Chopin and Mathieu Gerber. Sequential quasi Monte Carlo. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 77(3):509–579, 2015. doi: 10.1111/rssb.12084.
- Nicolas Chopin and Mathieu Gerber. Sequential quasi-Monte Carlo: Introduction for non-experts, dimension reduction, application to partly observed diffusion processes, 2017. Proceedings of the 2016 MCQMC Conference; arXiv:1706.05305.
- Nicolas Chopin and Omiros Papaspiliopoulos. *An Introduction to Sequential Monte Carlo*. Springer Series in Statistics. Springer, 2020.
- Pierre Del Moral. Non-linear filtering: interacting particle resolution. *Markov processes and related fields*, 2(4):555–581, 1996.
- Pierre Del Moral. *Feynman-Kac Formulae: Genealogical and Interacting Particle Systems with Applications*. Probability and Its Applications. Springer, 2004.
- Pierre Del Moral and Alice Guionnet. Central limit theorem for nonlinear filtering and interacting particle systems. *The Annals of Applied Probability*, 9(2):275–297, 1999. doi: 10.1214/aoap/1029962742.
- Pierre Del Moral, Arnaud Doucet, and Sumeetpal S. Singh. Forward smoothing using sequential monte carlo, 2010. arXiv:1012.5390.

- Pierre Del Moral, Arnaud Doucet, and Ajay Jasra. On adaptive resampling procedures for sequential Monte Carlo methods. *Bernoulli*, 18(1):252–278, 2012. doi: 10.3150/10-BEJ335.
- Luc Devroye. *Non-Uniform Random Variate Generation*. Springer, New York, 1986.
- Arnaud Doucet, Nando de Freitas, Kevin Murphy, and Stuart Russell. Rao-Blackwellised particle filtering for dynamic Bayesian networks. In *Proceedings of the Sixteenth Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 176–183, 2000.
- Samuel Duffield. ghq: Gauss–Hermite quadrature in JAX, 2024. URL <https://github.com/SamDuffield/ghq>. Accessed 11 September 2026.
- Samuel Duffield and Sumeetpal S. Singh. Online particle smoothing with application to map-matching. *IEEE Transactions on Signal Processing*, 70:497–508, 2022.
- Samuel Duffield and Ryan Tan. cuthberto-carlos: factorial state-space predictions for the 2026 FIFA World Cup, 2026. URL <https://github.com/state-space-models/cuthberto-carlos>. Apache-2.0 licence; version 1.0.0, commit 336842f; accessed 11 September 2026.
- Samuel Duffield, Samuel Power, and Lorenzo Rimella. A state-space perspective on modelling and inference for online skill rating. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 2024. doi: 10.1093/jrssc/qlae035. arXiv:2308.02414.
- Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. Arco Publishing, New York, 1978.
- FIFA. Fifa/coca-cola men’s world ranking. Wikipedia: The Free Encyclopedia, 2026. URL https://en.wikipedia.org/wiki/FIFA_Men%27s_World_Ranking. Ranking of 20 July 2026; accessed 9 September 2026.
- Mathieu Gerber. Safety of particle filters: Some results on the time evolution of particle filter estimates, 2026. arXiv:2503.21334, last revised January 2026.
- Zoubin Ghahramani and Michael I. Jordan. Factorial Hidden Markov Models. In *Advances in Neural Information Processing Systems 8 (NIPS 1995)*, pages 523–529. MIT Press, 1995.
- Zoubin Ghahramani and Michael I. Jordan. Factorial hidden Markov models. *Machine Learning*, 29(2):245–273, 1997.

- Simon J. Godsill, Arnaud Doucet, and Mike West. Monte Carlo smoothing for nonlinear time series. *Journal of the American Statistical Association*, 99(465):156–168, 2004.
- Neil J. Gordon, David J. Salmond, and Adrian F. M. Smith. Novel approach to nonlinear/non-Gaussian Bayesian state estimation. *IEEE Proceedings F (Radar and Signal Processing)*, 140(2):107–113, 1993. doi: 10.1049/ip-f-2.1993.0015.
- John H. Halton. On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. *Numerische Mathematik*, 2(1):84–90, 1960. doi: 10.1007/BF01386213.
- Chris H. Hamilton and Andrew Rau-Chaplin. Compact Hilbert indices for multi-dimensional data. In *Proceedings of the First International Conference on Complex, Intelligent and Software Intensive Systems*, 2008.
- Stephen Joe and Frances Y. Kuo. Constructing Sobol’ sequences with better two-dimensional projections. *SIAM Journal on Scientific Computing*, 30(5):2635–2654, 2008. doi: 10.1137/070709359.
- Rudolph E. Kalman and Richard S. Bucy. New results in linear filtering and prediction theory. *Journal of Basic Engineering*, 83(1):95–108, 1961. doi: 10.1115/1.3658902.
- Dimitris Karlis and Ioannis Ntzoufras. Analysis of sports data by using bivariate Poisson models. *Journal of the Royal Statistical Society Series D: The Statistician*, 52(3):381–393, 2003.
- Alexander Keller, Carsten Wächter, and Nikolaus Binder. Rendering along the Hilbert curve. In Zdravko Botev, Alexander Keller, Christiane Lemieux, and Bruno Tuffin, editors, *Advances in Modeling and Simulation: Festschrift for Pierre L’Ecuyer*, pages 319–332. Springer, 2022.
- Alexander Keller, Carsten Wächter, and Nikolaus Binder. Quasi-Monte Carlo algorithms (not only) for graphics software, 2023. arXiv:2307.15584.
- Mike Klaas, Nando de Freitas, and Arnaud Doucet. Toward practical N^2 Monte Carlo: the marginal particle filter. In *Proceedings of the Twenty-First Conference on Uncertainty in Artificial Intelligence*, UAI’05, pages 308–315, 2005.
- Pierre L’Ecuyer, Christian Lécot, and Bruno Tuffin. Randomized quasirandom methods for particle filters. In *Proceedings of the Winter Simulation Conference*, pages 952–960, 2006.

- Fredrik Lindsten and Thomas B. Schön. On the use of backward simulation in particle Markov chain Monte Carlo methods, 2011. arXiv:1110.2873.
- Fredrik Lindsten, Thomas B. Schön, Peter Bunch, and Simon J. Godsill. A backward-simulation based Rao-Blackwellized particle smoother for conditionally linear Gaussian models. In *Proceedings of the 16th IFAC Symposium on System Identification (SYSID)*, pages 506–511, 2012.
- Jiří Matoušek. On the L2-discrepancy for anchored boxes. *Journal of Complexity*, 14(4):527–556, 1998. doi: 10.1006/jcom.1998.0489.
- Harald Niederreiter. *Random Number Generation and Quasi-Monte Carlo Methods*, volume 63 of *CBMS-NSF Regional Conference Series in Applied Mathematics*. SIAM, 1992.
- Jimmy Olsson and Johan Westerborn. Efficient particle-based online smoothing in general hidden Markov models: the PaRIS algorithm. *Bernoulli*, 23(3):1951–1996, 2017.
- Art B. Owen. Monte Carlo variance of scrambled net quadrature. *SIAM Journal on Numerical Analysis*, 34(5):1884–1910, 1997. doi: 10.1137/S0036142994273168.
- Art B. Owen. Scrambling Sobol’ and Niederreiter–Xing points. *Journal of Complexity*, 14(4):466–489, 1998. doi: 10.1006/jcom.1998.0587.
- George Poyiadjis, Arnaud Doucet, and Sumeetpal S. Singh. Particle approximations of the score and observed information matrix in state space models with application to parameter estimation. *Biometrika*, 98(1):65–80, 2011. doi: 10.1093/biomet/asq062.
- Herbert E. Rauch, F. Tung, and Charlotte T. Striebel. Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8):1445–1450, 1965. doi: 10.2514/3.3166.
- Hans Sagan. *Space-Filling Curves*. Springer, 1994.
- Simo Särkkä, Pete Bunch, and Simon J. Godsill. Rao-Blackwellized particle smoothers for conditionally linear Gaussian models. *IEEE Journal of Selected Topics in Signal Processing*, 6(5):485–496, 2012.
- Adam Šcibior, Vaden Masrani, and Frank Wood. Differentiable particle filtering without modifying the forward pass, 2021. arXiv:2106.10314.

Chris Snyder, Thomas Bengtsson, Peter Bickel, and Jeffrey Anderson. Obstacles to high-dimensional particle filtering. *Monthly Weather Review*, 136(12):4629–4640, 2008. doi: 10.1175/2008MWR2529.1.

Il'ya M. Sobol'. On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Computational Mathematics and Mathematical Physics*, 7(4):86–112, 1967. doi: 10.1016/0041-5553(67)90144-9.

Nick Whiteley. Stability properties of some particle filters. *The Annals of Applied Probability*, 23(5):1968–2005, 2013. doi: 10.1214/12-AAP894.